Práticas de Engenharia de Software Contínua × fatores de QVT

Achados da etapa 2: como CI, CD, DevOps, *on-call* e práticas afins aumentam ou reduzem os fatores de qualidade de vida no trabalho de quem produz software, segundo o texto completo dos artigos

32
candidatos da busca 1 (aprovados e "talvez" no abstract)

26
lidos no texto completo

22
com elo prática → fator

65
elos prática → fator, todos com citação conferida

> **De onde vêm os elos.** Cada elo foi extraído do **texto completo** do artigo com as regras da skill *analise-esc-fatores*: só conta o que sai dos dados do próprio estudo; recomendação e trabalho citado não contam. Toda citação foi conferida por script contra o texto do PDF. **O que falta:** 6 candidatos ficaram sem PDF (seção 5), a busca 2 (790 registros) ainda não foi triada e os artigos ainda não estão no app.

Pergunta. Como a adoção de práticas de Engenharia de Software Contínua (ESC) aumenta ou reduz cada fator de QVT levantado na etapa 1? O que se registra é só o **primeiro elo**: o que a prática faz com a presença do fator. Se isso faz bem ou mal vem da polaridade do fator na etapa 1, mostrada entre parênteses ao lado do nome: *fator −* prejudica o bem-estar e *fator +* favorece.

Como ler os efeitos. ↑ **aumenta** e ↓ **reduz** descrevem a presença do fator, não o bem-estar. Reduzir *Interrupções no trabalho* (fator −) é bom; aumentar *Relações no trabalho* (fator +) também. **Evidência:** D = empírica direta (os dados ligam prática e fator: fala de participante, antes/depois, modelo); I = empírica indireta (co-ocorrência, interpretação dos autores); F = inferida (conclusão da discussão, sem dado que ligue os dois).

Base. Busca no Scopus (CAPES) em 2026-11-05; triagem dos abstracts e leitura do texto completo em 2026-11-06; catálogo de 64 fatores congelado em 24/09/2026. Protocolo em docs/busca-esc-qvt.md; dados em docs/esc/.

1. O que o corpus da etapa 1 já dizia

Varredura do texto completo dos 65 artigos com fatores: 37 não mencionam nenhuma prática de ESC, 22 só de passagem (referência, cargo do participante, nome de ferramenta) e 6 tocam no elo, sempre como evidência secundária.

Artigo	O que diz sobre prática → fator
Trinkenreich et al. (2023)	*Postmortem* sem culpados e times multifuncionais levam à cultura generativa (Westrum/DORA), que reduz o *burnout* — mas a ligação prática → cultura é citada, não medida
Trinkenreich et al. (2025)	Recomenda *postmortems* sem culpados para reforçar a segurança psicológica e o pertencimento
Cerqueira et al. (2024)	Líder empático equilibra as escalas de *on-call*; *postmortem* honesto reduz a culpa — em recomendações apresentadas como além da análise
Jafarzadeh et al. (2025)	Ciclos de entrega contínua mantêm intensidade alta e sem pausa, caminho para o *burnout*
Kumar et al. (2025)	"Automatizar tudo para a entrega contínua" libera tempo para o trabalho que o desenvolvedor valoriza
Benlian et al. (2025)	Em projeto grande, atraso pequeno se propaga pelo *pipeline* de entrega contínua e gera estresse e sensação de fracasso

Piloto da extração pelo texto completo

Três desses artigos foram lidos inteiros com as regras da skill. Só um elo se sustenta: recomendação apresentada além dos dados, ou com a ligação vinda de trabalho citado, não conta.

> **Benlian et al. (2025)** — Entrega contínua **↑ aumenta** *desfecho:* estresse e sensação de fracasso. **Mecanismo:** em projeto grande, o *pipeline* de entrega contínua interliga o trabalho de muitos, e um atraso pequeno se propaga e é vivido como falha em escala maior. **Condições:** projetos ágeis de larga escala; em projeto pequeno o participante I30 não relata o efeito. **Evidência:** empírica direta.
> Supplementary Qualitative Insights (I15): "it creates extra stress because even minor delays have ripple effects on our continuous delivery pipeline and feel like a failure on a much larger scale".

Cerqueira et al. (2024) e Trinkenreich et al. (2023): nenhum elo. Westrum (2004) e o relatório DORA, citados por Trinkenreich, viram candidatos ao corpus.

2. Buscas, triagem e leitura

Busca	O que pega	Registros
1 — ESC × QVT	Práticas de ESC + termos de bem-estar **e dos fatores** (carga, pressão de prazo, interrupção, carga cognitiva, *toil*, horas de trabalho, segurança psicológica…) + quem produz software	187
2 — estudos empíricos de ESC	Estudos de adoção e desafios de CI/CD/DevOps com entrevista, *survey* ou estudo de caso, menos o que já cai na busca 1	790

Filtros: artigo, conferência, revisão ou capítulo; inglês ou português; sem Medicina e Enfermagem. Calibração com 12 artigos-semente: a união das buscas recupera 8. Os 4 que ficam fora não citam nenhuma prática de ESC no resumo e entram por *snowballing*.

Triagem dos abstracts da busca 1

Veredito	Registros
Aprovado	17
Talvez (precisa do texto completo)	15
Reprovado — ferramenta/técnica	68
Reprovado — fora de ES	43
Reprovado — sem elo	17
Reprovado — sem pessoas	10
Reprovado — secundário	9
Reprovado — sem elo: CI só como atributo do ágil; o estresse é atribuído às limitações do ágil	2
Reprovado — veículo: revista de prática (Dr. Dobb's); sem elo entre CI e pessoas	1
Reprovado — ferramenta/técnica: IA generativa, sem prática de ESC	1
Reprovado — ferramenta/técnica: estimativa com LLM, sem prática de ESC	1
Reprovado — sem elo: o efeito sobre equilíbrio trabalho-vida é da IA generativa, não de prática de ESC	1
Reprovado — sem elo: DevOps só como cargo em anúncios de vagas	1
Reprovado — sem elo: o efeito sobre carga cognitiva é das ferramentas de IA; CI/CD só como recomendação	1

Leitura do texto completo dos 32 candidatos

Os abstracts afirmavam 22 elos; o texto completo sustenta 65, em 22 artigos. Vários elos do abstract caíram (o efeito estava só no resumo, ou vinha de trabalho citado) e muitos novos apareceram nas falas, tabelas e comparações antes/depois.

Resultado	Artigos	Quais
Influencia fatores: aprovado com elo	22	**Ahmad et al. (2023)**, **Anandayuvaraj et al. (2026)**, **Andreyev (2025)**, **Asthana et al. (2023)**, **Claes et al. (2017)**, **Fluri et al. (2024)**, **Hemon et al. (2018)**, **Itkonen et al. (2016)**, **Jeyam et al. (2026)**, **Kafoe et al. (2026)**, **Kassab et al. (2026)**, **Kärpänoja et al. (2016)**, **Limoncelli (2018)**, **Parry et al. (2022)**, **Pinto et al. (2017)**, **Riedl et al. (2025)**, **Rossi et al. (2016)**, **Shetty et al. (2022)**, **Soltren e Mulligan (2023)**, **Souza e Silva (2017)**, **Valenzuela-Toledo et al. (2024)**, **Zhang et al. (2018)**
Aprovado, mas sem elo no texto	1	**Fluri et al. (2023)**
Reprovado na leitura	3	**Gummadi et al. (2026)**, **Lertpongrujikorn et al. (2026)**, **Tennakoon e Jayatissa (2025)**
Sem PDF	6	**Ferazza e Mersinas (2026)**, **Claassen e Boekhorst (2015)**, **Hemon-Hildgen et al. (2020)**, **Jothi et al. (2026)**, **Kirpitsas e Pachidis (2026)**, **Kulas et al. (2014)**

3. Os elos: prática × fator

Cada linha liga uma prática de ESC a um fator do catálogo (ou a um fator emergente, ou direto a um desfecho). ↑ aumenta, ↓ reduz, ↕ ambivalente; o número é a quantidade de artigos. A última coluna dá os artigos e, entre colchetes, a força da evidência (D, I, F).

Prática	Fator	↑	↓	↕	Artigos
Automação de testes	Manutenção do ambiente de desenvolvimento (fator −)	3			**Ahmad et al. (2023)** [D]; **Kärpänoja et al. (2016)** [D]; **Parry et al. (2022)** [D]
Automação de testes	Fadiga de alertas (fator emergente, Tarefa e carga de trabalho)	1			**Andreyev (2025)** [I]
Bots de desenvolvimento no pipeline de CI/CD	Carga cognitiva (fator −)	1			**Asthana et al. (2023)** [D]
Bots de desenvolvimento no pipeline de CI/CD	Carga de trabalho (fator −)			1	**Asthana et al. (2023)** [D]
Bots de desenvolvimento no pipeline de CI/CD	Conflito de papéis (fator −)	1			**Asthana et al. (2023)** [F]
Bots de desenvolvimento no pipeline de CI/CD	Documentação interna (fator +)	1			**Asthana et al. (2023)** [D]
Bots de desenvolvimento no pipeline de CI/CD	Integração de novatos (fator +)	1			**Asthana et al. (2023)** [D]
ESC/DevOps em geral	Desenho do trabalho (fator −, nome neutro*)	1			**Hemon et al. (2018)** [I]
Entrega contínua	Ansiedade de implantação (fator emergente, Técnico e ferramentas)		2		**Kärpänoja et al. (2016)** [D]; **Limoncelli (2018)** [F]
Entrega contínua	Conflito trabalho-família (fator −)		2		**Kärpänoja et al. (2016)** [I]; **Limoncelli (2018)** [F]
Entrega contínua	Carga de trabalho (fator −)			1	**Itkonen et al. (2016)** [D]
Entrega contínua	Desenho do trabalho (fator −, nome neutro*)	1			**Itkonen et al. (2016)** [D]
Entrega contínua	Interação com clientes (fator −, nome neutro*)		1		**Itkonen et al. (2016)** [I]
Entrega contínua	Relações no trabalho (fator +)	1			**Kärpänoja et al. (2016)** [I]
Entrega contínua	Risco de falha na implantação (fator emergente, Técnico e ferramentas)		1		**Itkonen et al. (2016)** [D]
Gestão de incidentes	Carga cognitiva (fator −)	1			**Shetty et al. (2022)** [D]
Gestão de incidentes	Carga de trabalho (fator −)		1		**Shetty et al. (2022)** [I]
Gestão de incidentes	Interrupções no trabalho (fator −)	1			**Shetty et al. (2022)** [D]
Implantação contínua	Carga cognitiva (fator −)	1			**Hemon et al. (2018)** [D]
Implantação contínua	Carga de trabalho (fator −)		1		**Zhang et al. (2018)** [D]
Integração contínua	Carga de trabalho (fator −)	2			**Pinto et al. (2017)** [D]; **Soltren e Mulligan (2023)** [F]
Integração contínua	Alinhamento do uso do tempo (fator +)		1		**Pinto et al. (2017)** [D]

Prática	Fator	↑	↓	↕	Artigos
Integração contínua	Confiança para alterar o código (fator emergente, Técnico e ferramentas)	1			**Pinto et al. (2017)** [D]
Integração contínua	Integração de novatos (fator +)		1		**Pinto et al. (2017)** [D]
Integração contínua	Manutenção do ambiente de desenvolvimento (fator −)		1		**Soltren e Mulligan (2023)** [I]
Integração contínua	Percepção de recompensa (fator emergente, Organizacional)	1			**Riedl et al. (2025)** [F]
Integração contínua	Pressão por manter o build verde (fator emergente, Tarefa e carga de trabalho)	1			**Pinto et al. (2017)** [D]
Monitoramento e observabilidade	Carga cognitiva (fator −)			1	**Jeyam et al. (2026)** [I]
On-call	Conflito trabalho-família (fator −)	1			**Jeyam et al. (2026)** [D]
Pipeline de CI/CD	Manutenção do ambiente de desenvolvimento (fator −)	3		1	**Parry et al. (2022)** [D]; **Pinto et al. (2017)** [D]; **Valenzuela-Toledo et al. (2024)** [I]; **Zhang et al. (2018)** [D]
Pipeline de CI/CD	Carga cognitiva (fator −)		1	1	**Fluri et al. (2024)** [D]; **Jeyam et al. (2026)** [I]
Pipeline de CI/CD	Interrupções no trabalho (fator −)	1	1		**Fluri et al. (2024)** [D]; **Kassab et al. (2026)** [F]
Pipeline de CI/CD	Ansiedade de implantação (fator emergente, Técnico e ferramentas)		1		**Fluri et al. (2024)** [D]
Pipeline de CI/CD	Desenho do trabalho (fator −, nome neutro*)	1			**Fluri et al. (2024)** [D]
Pipeline de CI/CD	*desfecho:* emoção negativa expressa nas discussões (sobretudo tristeza/frustração)	1			**Kassab et al. (2026)** [I]
Pipeline de CI/CD	*desfecho:* sentimento negativo expresso nas mensagens de commit	1			**Souza e Silva (2017)** [D]
Postmortem sem culpados	Carga de trabalho (fator −)			1	**Anandayuvaraj et al. (2026)** [I]
Postmortem sem culpados	Documentação interna (fator +)	1			**Anandayuvaraj et al. (2026)** [I]
Releases frequentes	Pressão de prazos (fator −)	1	1		**Hemon et al. (2018)** [D]; **Rossi et al. (2016)** [I]
Releases frequentes	Ansiedade de implantação (fator emergente, Técnico e ferramentas)		1		**Kärpänoja et al. (2016)** [D]
Releases frequentes	Carga cognitiva (fator −)		1		**Kärpänoja et al. (2016)** [D]
Releases frequentes	Conflito trabalho-família (fator −)		1		**Claes et al. (2017)** [D]
Responsabilidade de operação pelo time	Conflito trabalho-família (fator −)	1			**Jeyam et al. (2026)** [D]
Responsabilidade de operação pelo time	Desenho do trabalho (fator −, nome neutro*)	1			**Kärpänoja et al. (2016)** [I]
Responsabilidade de operação pelo time	Pressão por atualização (fator −)	1			**Jeyam et al. (2026)** [D]

Prática	Fator	↑	↓	↕	Artigos
Site Reliability Engineering	Alinhamento do uso do tempo (fator +)	1			**Kafoe et al. (2026)** [D]
Site Reliability Engineering	Desenho do trabalho (fator –, nome neutro*)	1			**Kafoe et al. (2026)** [D]
Site Reliability Engineering	Desenvolvimento de competências (fator emergente, Individual)			1	**Kafoe et al. (2026)** [D]
Site Reliability Engineering	Insegurança no emprego (fator –)	1			**Kafoe et al. (2026)** [D]
Site Reliability Engineering	Senso de realização (fator +)			1	**Kafoe et al. (2026)** [D]
Times multifuncionais Dev–Ops	Apoio do gestor (fator +)		1		**Hemon et al. (2018)** [I]
Times multifuncionais Dev–Ops	Conflito de papéis (fator –)	1			**Hemon et al. (2018)** [D]
Times multifuncionais Dev–Ops	Reconhecimento no trabalho (fator +)			1	**Hemon et al. (2018)** [D]
Times multifuncionais Dev–Ops	Relações no trabalho (fator +)	1			**Hemon et al. (2018)** [D]

* **Desenho do trabalho**: a descrição é neutra (variedade, autonomia, retorno), mas na etapa 1 só aparece o caso ruim (4 artigos, todos negativos). Nos elos desta etapa, "aumenta" quer dizer trabalho **mais bem desenhado** — o que atenua o fator negativo, ao contrário do que a polaridade sugere.

* **Interação com clientes**: o nome é neutro, mas a descrição e a polaridade da etapa 1 (8 negativos) tratam do desgaste com o cliente. Nos elos desta etapa o efeito se refere a esse desgaste: "reduz" quer dizer menos retrabalho e espera pelo cliente.

Por fator

A mesma informação vista do lado dos fatores: quais práticas mexem em cada um.

Fator	Práticas que mexem nele
Manutenção do ambiente de desenvolvimento (fator –)	Automação de testes ↑ ×3; Pipeline de CI/CD ↑ ×3; Integração contínua ↓; Pipeline de CI/CD ↕
Carga cognitiva (fator –)	Bots de desenvolvimento no pipeline de CI/CD ↑; Pipeline de CI/CD ↓; Implantação contínua ↑; Monitoramento e observabilidade ↕; Pipeline de CI/CD ↕; Releases frequentes ↓; Gestão de incidentes ↑
Carga de trabalho (fator –)	Postmortem sem culpados ↕; Bots de desenvolvimento no pipeline de CI/CD ↕; Entrega contínua ↕; Integração contínua ↑ ×2; Gestão de incidentes ↓; Implantação contínua ↓
Conflito trabalho-família (fator –)	Releases frequentes ↓; On-call ↑; Responsabilidade de operação pelo time ↑; Entrega contínua ↓ ×2
Desenho do trabalho (fator –, nome neutro*)	Pipeline de CI/CD ↑; ESC/DevOps em geral ↑; Entrega contínua ↑; Site Reliability Engineering ↑; Responsabilidade de operação pelo time ↑
Ansiedade de implantação (fator emergente, Técnico e ferramentas)	Pipeline de CI/CD ↓; Entrega contínua ↓ ×2; Releases frequentes ↓
Interrupções no trabalho (fator –)	Pipeline de CI/CD ↓; Pipeline de CI/CD ↑; Gestão de incidentes ↑
Alinhamento do uso do tempo (fator +)	Site Reliability Engineering ↑; Integração contínua ↓
Conflito de papéis (fator –)	Bots de desenvolvimento no pipeline de CI/CD ↑; Times multifuncionais Dev–Ops ↑
Documentação interna (fator +)	Postmortem sem culpados ↑; Bots de desenvolvimento no pipeline de CI/CD ↑
Integração de novatos (fator +)	Bots de desenvolvimento no pipeline de CI/CD ↑; Integração contínua ↓
Pressão de prazos (fator –)	Releases frequentes ↑; Releases frequentes ↓

Fator	Práticas que mexem nele
Relações no trabalho (fator +)	Times multifuncionais Dev–Ops ↑; Entrega contínua ↑
Apoio do gestor (fator +)	Times multifuncionais Dev–Ops ↓
Confiança para alterar o código (fator emergente, Técnico e ferramentas)	Integração contínua ↑
Desenvolvimento de competências (fator emergente, Individual)	Site Reliability Engineering ↕
Fadiga de alertas (fator emergente, Tarefa e carga de trabalho)	Automação de testes ↑
Insegurança no emprego (fator −)	Site Reliability Engineering ↑
Interação com clientes (fator −, nome neutro*)	Entrega contínua ↓
Percepção de recompensa (fator emergente, Organizacional)	Integração contínua ↑
Pressão por atualização (fator −)	Responsabilidade de operação pelo time ↑
Pressão por manter o build verde (fator emergente, Tarefa e carga de trabalho)	Integração contínua ↑
Reconhecimento no trabalho (fator +)	Times multifuncionais Dev–Ops ↕
Risco de falha na implantação (fator emergente, Técnico e ferramentas)	Entrega contínua ↓
Senso de realização (fator +)	Site Reliability Engineering ↕
desfecho: emoção negativa expressa nas discussões (sobretudo tristeza/frustração)	Pipeline de CI/CD ↑
desfecho: sentimento negativo expresso nas mensagens de commit	Pipeline de CI/CD ↑

4. Achados por prática

Cada elo com o mecanismo e as condições segundo o artigo e a citação do texto completo. O nome do artigo leva à ficha dele (seção 5).

Pipeline de CI/CD

12 elos em 8 artigo(s)

↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)
Fluri et al. (2024) *— On the importance of CI/CD practices for database applications*
Mecanismo: antes, mudar código direto em produção sem validação gerava medo de implantar; o feedback imediato do CI e os testes automáticos do pipeline dão confiança de que a mudança funciona antes de chegar à produção. **Condições:** o estresse de implantar foi relatado antes só no UC3-RET, que alterava código direto em produção; o aumento de confiança foi relatado nos três casos. **Evidência:** empírica direta.
Seção 6.3 (UC3-RET, antes): “In UC3-RET, developers reported high stress levels when altering code in production directly. Their goal was to reduce this stress, so changes can be deployed with confidence.”; Seção 7.1 (depois): “The instant feedback the developers get from the CI increases trust in deploying features to production.”; “In all three use cases, developer confidence in adding changes increased because they could rely on the automated pipeline to test them thoroughly”

↓ **reduz** Carga cognitiva (fator −)
Fluri et al. (2024) *— On the importance of CI/CD practices for database applications*
Mecanismo: o pipeline automatiza integração, release e implantação que antes eram manuais e testa cada mudança, então desenvolvedores e coordenadores de release deixam de manter várias features em paralelo prontas para consertar e passam a confiar no pipeline para implantar. **Condições:** relatado nos três casos; no UC3-RET a troca para controle de versão e novo processo de release trouxe uma curva de aprendizado íngreme na transição. **Evidência:** empírica direta.
Seção 5.2 (UC2-BE, antes): “Having several features in the pipeline in parallel also increased the cognitive load on the developers because they always had to be prepared to fix packages as they got promoted.”; Seção 6.3 (depois): “The feedback from the three use cases after the pipeline implementation revealed that developers and release coordinators experience a lighter cognitive load in daily work.”; “They can now concentrate on development and trust the pipeline to deploy the changes correctly.”; Seção 7.1: “The interviews showed that developers now have fewer parallel tasks and can test features independently”; Seção 6.3 (UC3-RET): “the usage of a version control system together with a new release process introduces a steep learning curve.”

↕ **ambivalente** Carga cognitiva (fator −)
Jeyam et al. (2026) *— "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI*
Mecanismo: pipelines heterogêneos entre times e etapas de implantação ainda manuais obrigam a trocar de tecnologia e de modo de trabalho, o que exige troca de contexto mental; pipelines padronizados e automatizados aliviam essa carga. **Condições:** aumenta quando a maturidade dos pipelines varia entre times e parte da implantação é manual; reduz com padronização das tecnologias de pipeline e automação de build, teste e implantação. **Evidência:** empírica indireta.
Seção 4.2 (Application Manager, Nr. 21): “They have also set up pipelines and mapped out the entire chain. Some are very advanced, and some are still in the Stone Age. They do everything by hand”; Seção 4.2: “still required manual deployment, which can lead to mental context shifts”; Seção 4.2 (SE (Dev), Nr. 2): “This is another manual; it is not automatized. Someone has to log in and perform this step in order to proceed to the next step”; Seção 4.3 (SE (Dev), Nr. 20): “It would be easier if you had everything standardized”; “and didn’t have to switch between pipeline technologies.”; Seção 4.3: “standardization alleviates the cognitive load on DevOps members”

↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)
Fluri et al. (2024) *— On the importance of CI/CD practices for database applications*
Mecanismo: o pipeline permite que cada desenvolvedor promova e implante a própria mudança e teste a feature de forma independente, sem passar por um coordenador central de release (mais autonomia e entrega do trabalho inteiro). **Condições:** depende de a organização dar permissão de implantação e confiar nos desenvolvedores: só no UC1-DWH todos passaram a implantar em produção (1 para 30 pessoas); no UC2-BE a produção seguiu restrita ao desenvolvedor líder por exigência de VPN do cliente. **Evidência:** empírica direta.
Seção 7.1: “The ability to deploy features for all developers also removed the release coordinator as a single point of failure and dependency for the teams in two use cases.”; Seção 6.2: “only UC1-DWH enabled all developers to execute production deployments”; Seção 7.1: “the need to trust all developers to promote their changes through the pipeline without a central coordinator.”; Tabela 4: “People able to deploy to production 1 30 1 1 35 35”

↓ **reduz** Interrupções no trabalho (fator −)
Fluri et al. (2024) *— On the importance of CI/CD practices for database applications*
Mecanismo: com a implantação manual, o coordenador de release voltava ao desenvolvedor a cada script que falhava, obrigando-o a largar o que fazia; com o pipeline o erro aparece cedo e o troubleshooting de integração e implantação deixa de interromper o trabalho. **Condições:** comparação antes/depois nos casos com coordenador central de release (UC1-DWH e UC2-BE) e relato de mais fluxo nos três casos. **Evidência:** empírica direta.
Seção 6.3 (UC1-DWH e UC2-BE, antes): “manual deployments involved a high level of context switching, because the release coordinator needed to get back to the developers if installations failed, causing them to pause their current work for bug fixes.”; Seção 5.2 (UC2-BE, antes): “Not getting instant feedback if the change scripts worked caused a lot of context switching when the change coordinator needed information about a failed script.”; Seção 6.3 (depois): “They also mentioned that their daily work had a lot more flow since they were not interrupted by integration and deployment troubleshooting as much as before”

↑ **aumenta** Interrupções no trabalho (fator −)
Kassab et al. (2026) *— The Emotional Cost of Technical Debt: Quantifying Developer Frustration Across Debt-Related Concern Themes*
Mecanismo: falhas de CI e de testes quebram o ciclo de feedback e o ritmo de desenvolvimento, forçando troca de contexto repetida e trabalho reativo. **Condições:** pipeline com dívida (builds quebrados, testes flaky); interpretação dos autores sobre os temas de NEI alto, sem dado que meça interrupção. **Evidência:** inferida.
Seção 5.2: “The high-NEI themes identified in our study—particularly CI/test failures and observability or configuration issues—are precisely the kinds of concerns that interrupt development rhythm and force repeated context switching”; Seção 5.1: “Operational and infrastructure-related debt frequently disrupt feedback loops (e.g., broken pipelines, opaque logs, misconfigurations)”

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)
Pinto et al. (2017) *— Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*
Mecanismo: usar CI exige configurar e manter o ambiente de build; quando o build falha no servidor e não localmente fica difícil depurar, e testes instáveis frustram. **Condições:** parte dos respondentes considera o custo de configuração administrável; pesa mais em testes distribuídos ou com múltiplos componentes. **Evidência:** empírica direta.
Seção IV.B: “we found that 31 respondents are having a hard time configuring the build environment”; “Some extra overhead and complexity for setting them up / maintaining the configuration. Not usually a big problem”; “If build fails it may be hard to debug it, especially if the problem doesn’t occur in the local environment”; “Tests can sometimes be flaky, which is frustrating”

↕ **ambivalente** Manutenção do ambiente de desenvolvimento (fator −)
Zhang et al. (2018) *— One size does not fit all: An empirical study of containerized continuous deployment workflows*
Mecanismo: o pipeline automatiza a configuração de ambientes e libera tempo para desenvolver, mas ele próprio exige configuração difícil, builds lentos que travam o trabalho e ciclos de tentativa e rebuild quando quebra. **Condições:** depende do modo de adoção: o workflow via CI (mais testes, mais configurável) tem mais builds com erro e exige mais manutenção e experiência; o auto-build do Docker Hub é mais simples e de manutenção menor, porém com latência maior; a latência de build cresce com o tempo; vários respondentes trocaram de workflow por ser difícil de configurar e manter. **Evidência:** empírica direta.
Seção 3.2, Tabela 1: “Lets us spend less time on maintenance and configuration 9 6.4%”; [M7] (R64): “to optimize time, focusing on developing the actual application”; Seção 3.4 [N1] (R4): “one dockerfile takes more than 2 hours to build and timeouts”; Seção 3.4: “The CD processing speed will affect the software release and developers’ work efficiency.”; [N2] (R109): “sometimes, circleCI config and setup is pain. Docs sometimes doesn’t help”; Seção 3.5 [B1] (R119): “the old CD pipeline is a little harder to setup. It was necessary to write several scripts to get everything working properly. The new CD pipeline is easier to setup and maintain”; Seção 5.2: “More testing means higher reliability, but also more errored builds, i.e., more maintenance.”

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)
Parry et al. (2022) *— Surveying the Developer Experience of Flaky Tests*
Mecanismo: diferenças entre a máquina local e o ambiente remoto de CI, e dependências externas baixadas no build, fazem testes falharem só na CI, o que leva a reexecutar builds e investigar falhas que não são do código. **Condições:** quando o ambiente de CI difere do local (padrões da JVM, latência de sistema de arquivos, concorrência) ou depende de serviços e bibliotecas externos. **Evidência:** empírica direta.
RQ2, SQ4: “developers strongly agree with the notion that flaky tests hinder CI”; RQ3, SQ6t2 (P21): “environmental differences in local vs CI like different Java Virtual Machine (JVM) defaults.”; SQ6t2: “Almost all the responses in this theme made reference to CI.”; Tabela 1, SQ11t7 (P31): “Recently, seen a lot of flaky tests when running CI on Azure due to failures to download libraries”

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)
Valenzuela-Toledo et al. (2024) *— The Hidden Costs of Automation: An Empirical Study on GitHub Actions Workflow Maintenance*
Mecanismo: os arquivos de workflow que automatizam build, teste e deploy não ficam estáveis: exigem mudança manual recorrente para consertar defeitos, atualizar dependências e melhorar o próprio CI/CD, inclusive quando uma alteração de infraestrutura faz todos os pull requests falharem. **Condições:** projetos open source grandes e maduros no GitHub Actions; o esforço é medido por commits, que os autores admitem ser proxy imperfeito (mudança pequena pode custar muito esforço cognitivo); a manutenção recai sobre poucos desenvolvedores (cerca de 5%), que também mexem em código de produção e teste. **Evidência:** empírica indireta.
Seção IV-B, Finding 2: “In the multi-workflow group, a mean of 8.2% of the workflow files undergo at least one monthly change”; Seção IV-D (1.b): “involves reverting a change to Debian 11 (bullseye) that caused all pull requests to fail”; Seção V (a): “Nonetheless, they are far from being stable, which confirms our hypothesis that manual effort is needed for maintaining workflow files over time”; Seção V (b): “thorough investigation of commits that contribute to the logical coupling reveals that they are primarily updated for CI/CD improvements and bug fixes”; Seção VIII: “However, our research reveals that implementing and maintaining these workflow files is not without additional costs”

↑ **aumenta** *desfecho:* sentimento negativo expresso nas mensagens de commit
Souza e Silva (2017) *— Sentiment Analysis of Travis CI Builds*
Mecanismo: o build quebrado no servidor de CI desperta frustração que aparece no commit seguinte, e quebras repetidas antes de conseguir consertar levam o desenvolvedor a odiar o bug, num ciclo de realimentação entre sentimento e quebra. **Condições:** commit imediatamente após um build quebrado (Travis CI, projetos de código aberto); efeito estatisticamente significativo mas muito pequeno, medido por análise de sentimento automática sujeita a erro e sarcasmo. **Evidência:** empírica direta.
Seção III.B: “The proportion of builds with more extreme negative scores is higher after broken builds”; “although the effect size is small (Cliff’s delta = −0.014)”; Seção III.B: “Commits following a build breakage tend to be more negative and less positive, although the effect size is small.”; Seção III.B (Tabela III, commit e): “multiple builds failed before the developer managed to fix bug #320, causing him to hate the bug”; Seção III.D: “We also found that broken builds are more likely to be followed by negative sentiment, creating a feedback loop.”; “the actual effect sizes may be even smaller than reported”

↑ **aumenta** *desfecho:* emoção negativa expressa nas discussões (sobretudo tristeza/frustração)
Kassab et al. (2026) *— The Emotional Cost of Technical Debt: Quantifying Developer Frustration Across Debt-Related Concern Themes*
Mecanismo: a dívida acumulada no pipeline e na suíte de testes (testes flaky, builds instáveis, falhas dependentes de plataforma) mina a confiança na automação, e as discussões sobre ela carregam mais emoção negativa que as de dívida de API ou de implementação. **Condições:** pipeline e suíte de testes com dívida (testes flaky, build instável); comparação relativa a outros temas de dívida, não a projetos sem CI; efeito estatisticamente significativo mas pequeno, sem pretensão causal; três projetos Apache de código aberto, emoção inferida por classificador a partir do texto. **Evidência:** empírica indireta.
Seção 4.1: “Topic 4 focuses on testing and CI/CD infrastructure debt”; “flaky tests, unstable build pipelines, and platform-dependent failures that undermine trust in automation”; Seção 4.3: “Topic 4 (Testing and CI/CD) also shows elevated NEI”; “Conversely, Topics 3 and 4 lean neutral-to-negative (EBI ≤ 0), consistent with frustration associated with monitoring failures and unstable automation”

Entrega contínua

9 elos em 3 artigo(s)

↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)
Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*
Mecanismo: cada mudança é construída, testada e implantada automaticamente no ambiente de teste, o que dá confiança de que vai funcionar em produção; e como corrigir e implantar a correção é simples e imediato, bugs e implantações deixam de gerar estresse. **Condições:** relatado nos três projetos, todos com pipeline automatizado e implantação diária possível; os autores ressalvam que as implantações precisam ser também frequentes (ver o registro de Releases frequentes) e que pessoas mais conservadoras poderiam sentir mais estresse com a mudança. **Evidência:** empírica direta.
Seção 4.1: “Because the software was built, tested, and deployed to test environment automatically after each change”; “team had high trust for the application to work expectedly in production”; “they even felt that the shelter brought by the tests was so precious they would like to spend even more time writing tests”; Seção 4.2: “In all cases, the developers did not feel any stress due to bugs, because fixing and deploying the fixes to production was so straightforward.”; Seção 5: “feel stress over deployments or bugs that would appear in the production, because they were always able to deploy fixes to production – immediately, if needed.”

↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)
Limoncelli (2018) *— SQL is no excuse to avoid DevOps*
Mecanismo: a implantação manual e rara é vivida como risco pessoal (dados da empresa e o próprio emprego); automatizada e testada muitas vezes antes de produção, ela vira evento rotineiro e sem incidente. **Condições:** a frequência maior só ajuda se o processo for automatizado e testado; com o processo manual, mais implantações significariam mais estresse. **Evidência:** inferida.
Relato do amigo: “these deployments are risky. Every time we do one I risk the company’s data and, to be honest, my job.”; “Doing this every few months is painful and the number-one source of stress around here.”; Summary: “Automating releases liberates us. It turns a worrisome, stressful, manual upgrade process into a regular event that happens without incident.”; “It reduces business risk but, more importantly, creates a more sustainable workplace.”

↕ **ambivalente** Carga de trabalho (fator −)
Itkonen et al. (2016) *— Perceived Benefits of Adopting Continuous Delivery Practices*
Mecanismo: durante a adoção, montar ferramentas e práticas novas exigiu esforço grande, às vezes acima do previsto, por ter de aprender ferramentas desconhecidas; depois, a automação de implantação encurtou implantações que antes podiam tomar a noite inteira da equipe do cliente. **Condições:** aumenta na transição (aprendizado das ferramentas) e reduz depois que a automação está pronta; a redução é relatada pelo lado do cliente, que operava a implantação; as implantações em produção continuam fora do expediente e ainda exigem janela de manutenção. **Evidência:** empírica direta.
Seção 3.1.1: “establishing the new tools and practices required substantial effort, sometimes more than expected, due to learning unfamiliar tools and techniques”; Seção 3.1.2: “Prior to the improvements, deployments could take from a few hours to the entire evening and night”; Seção 3.1.2: “The situation has improved by deployment automation”; Seção 3.1.1: “have to take place outside of office hours”

↓ **reduz** Conflito trabalho-família (fator −)
Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*
Mecanismo: com implantação automatizada e frequente, implantar vira rotina que não exige ninguém acompanhando nem acordado; não há janela de manutenção à noite ou no fim de semana, e as implantações cabem no expediente sem hora extra. **Condições:** projetos pequenos com um único pacote implantável e implantação em produção em um passo; um time chegou a agendar a implantação automática para a noite, decidida durante o dia. **Evidência:** empírica indireta.
Seção 4.1: “When deliveries are frequent and automated, the deployment becomes a normal process which does not require anyone to be monitoring or even awake at the time.”; “Deployments were done during standard working hours and required no overtime or particular scheduling.”; Seção 5: “They did not work overtime”

↓ **reduz** Conflito trabalho-família (fator −)

Limoncelli (2018) *— SQL is no excuse to avoid DevOps*

Mecanismo: com mudança de esquema automatizada e código compatível com vários esquemas, a implantação deixa de exigir parada da aplicação e janela de manutenção negociada entre times, que acabava caindo no fim de semana; o DBA deixa de participar pessoalmente de cada atualização. **Condições:** depende de automatizar o esquema e de reduzir o custo fixo da implantação; sem isso, o amigo do autor prevê que releases semanais acabariam com os fins de semana e o autor chama de irresponsável aumentar a frequência sem baixar esse custo. **Evidência:** inferida.

Relato do amigo: “The process takes many hours, so we tend to do it on the weekend... which I hate.”; “Just scheduling such an event takes weeks of negotiation. We usually lose the negotiation, which is why we end up doing it on the weekend.”; Technique 1: “This should bring new confidence to the process, reduce the risk of schema upgrades, and decouple the DBAs’ personal involvement in upgrades. They will appreciate getting their weekends back.”; Technique 2: “While in the past upgrades required finding a maintenance window compatible with multiple teams, now the process is decoupled and all parties can work in a coordinated way but not in lockstep.”

↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)

Itkonen et al. (2016) *— Perceived Benefits of Adopting Continuous Delivery Practices*

Mecanismo: a automação de implantação e de configuração tira do desenvolvedor tarefas manuais repetitivas, e o retorno sobre o que foi implantado chega bem mais rápido; o artigo liga as duas coisas ao aumento de satisfação (aqui 'aumenta' quer dizer melhora do desenho: menos tarefa mecânica, mais retorno do resultado). **Condições:** melhorias propostas de baixo para cima por desenvolvedores incomodados com tarefas manuais e decididas coletivamente; parte da automação é de configuração de servidores (infraestrutura como código), que o artigo não separa da entrega contínua. **Evidência:** empírica direta.

Seção 3.1.1: “Not only has the automation of tasks decreased the amount of repetitive work, such as manual”; Seção 3.7: “of repetitive, menial tasks. Interviewees reported that many of the ideas that led to changes simply stemmed from individual developers being annoyed with having to perform some task manually”; Seção 3.7: “These kinds of bottom-up changes have resulted in increased developer satisfaction”; Seção 3.2.1: “the feedback on deployed changes, which now arrives notably faster than five years ago”

↓ **reduz** Interação com clientes (fator −, nome neutro*)

Itkonen et al. (2016) *— Perceived Benefits of Adopting Continuous Delivery Practices*

Mecanismo: o pipeline de implantação permite prototipar continuamente e levar cada mudança ao ambiente de teste do cliente, então a divergência de entendimento do requisito aparece cedo e o trabalho feito contra especificação mal interpretada deixa de virar retrabalho; o cliente também ficou mais fácil de consultar e a espera por ele caiu. O contato com o cliente aumentou; o que diminuiu foi o desgaste que o fator descreve. **Condições:** junto com práticas de colaboração adotadas na mesma transição (diálogo contínuo sobre especificações, desenvolvimento colocalizado, sistema de tickets comum, mensagem instantânea), que o artigo não separa do pipeline; o feedback do usuário continua limitado pela agenda do cliente. **Evidência:** empírica indireta.

Seção 2.4.1 (estado inicial): “lead to considerable amounts of wasted work”; Seção 3.1.1: “specs is now largely avoided through continuous”; Seção 3.7: “the continuous prototyping, enabled by the deployment pipeline and related automation, allows developers to get faster validation of their interpretation of the requirements”; Seção 3.2.1: “Developers also have a much lower threshold for contacting the customer”; Seção 3.1.1: “the time spent waiting for deployment decisions, feedback, and customer”; Seção 3.8: “User feedback can be gained more quickly, but is limited by the schedules of the customer stakeholders.”

↑ **aumenta** Relações no trabalho (fator +)

Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*

Mecanismo: como todo merge no ramo principal precisa estar pronto para produção e cada um testa a própria implementação, os desenvolvedores passam a confiar que as mudanças dos colegas também são implantáveis, o que forma um time unido que responde junto pelo produto. **Condições:** depende de o time assumir a responsabilidade pela qualidade e pelos testes; os autores notam que os desenvolvedores da Solita são abertos a mudança. **Evidência:** empírica indireta.

Seção 4.1: “they also had to trust their colleges that their changes are deployable, too. This created a unified team that was ready to take responsibility for the whole product.”; “When teams had more responsibility, they clearly did not consider themselves as just programmers but rather as full members in a solid team.”

↓ **reduz** Risco de falha na implantação (fator emergente, Técnico e ferramentas)
Itkonen et al. (2016) — *Perceived Benefits of Adopting Continuous Delivery Practices*
Mecanismo: builds independentes de ambiente, teste automático de toda mudança, menos passos para acessar os ambientes do cliente e ambientes parecidos com produção diminuem a chance de a implantação falhar e o trabalho extra que a falha exige; os desenvolvedores atribuem a isso boa parte da queda do estresse. **Condições:** organização ainda em adoção de CD, com monitoramento de produção incipiente. **Evidência:** empírica direta.
Seção 3.4: “Four factors were identified as having decreased release and deployment risk significantly”; Seção 3.4: “the automatic testing of all changes has reduced the risk of uncovering bugs in the applications after production deployments”; Seção 3.7: “Much of the reduction in stress levels results from the reduced risk of failing deployments and the additional work that those require”

Integração contínua

8 elos em 3 artigo(s)

↓ **reduz** Alinhamento do uso do tempo (fator +)
Pinto et al. (2017) — *Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*
Mecanismo: a disciplina da CI desvia a atenção do código da aplicação para o código do sistema de build, e o engenheiro de software passa a fazer trabalho de infraestrutura, que considera bem menos interessante. **Condições:** relato de um respondente. **Evidência:** empírica direta.
Seção IV.B: “instead of only focusing on the code of your application, you also focus on the code of your build system”; “becomes an infrastructure engineer, which is much less interesting”

↑ **aumenta** Carga de trabalho (fator −)
Pinto et al. (2017) — *Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*
Mecanismo: manter os builds passando exige esforço adicional de compartilhar, documentar e ajudar a resolver problemas, o que segundo os respondentes pede mais gente. **Evidência:** empírica direta.
Seção IV.B: “additional effort was also evidenced by 3 other respondents”; “Need more people sharing / Documenting and helping resolve problems / Making sure builds pass”

↑ **aumenta** Carga de trabalho (fator −)
Soltren e Mulligan (2023) — *Enabling Pre-Merge CI on your TV*
Mecanismo: tornar obrigatória a aprovação no CI pré-merge acrescentou passos ao processo de quem faz commit sempre que o merge falha nas verificações. **Condições:** quando o merge falha nas verificações obrigatórias. **Evidência:** inferida.
corpo (desafios organizacionais): “CI checks resulted in additional process for committers when merges failed.”

↑ **aumenta** Confiança para alterar o código (fator emergente, Técnico e ferramentas)
Pinto et al. (2017) — *Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*
Mecanismo: o build automatizado com testes dá segurança de que a mudança não quebrou o comportamento existente, e tira o medo de quebrar o build num ramo. **Condições:** depende da cobertura de testes; 25 respondentes apontam falsa sensação de confiança quando se confia cegamente nos testes. **Evidência:** empírica direta.
Seção IV.A: “continuous integration gives more confidence to perform required code changes”; Seção III.B: “CI is there for you not to be afraid for broken builds in a branch”; Seção IV.B: “Another recurring problem is the false sense of confidence”

↓ **reduz** Integração de novatos (fator +)
Pinto et al. (2017) — *Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*
Mecanismo: os problemas técnicos de configuração e o fluxo de CI pouco explicado dificultam a entrada de novatos e contribuidores casuais, que não entendem o que a CI faz e só descobrem o fluxo depois de submeter o patch. **Condições:** projetos open source sem orientação ou documentação para quem chega. **Evidência:** empírica direta.
Seção IV.B: “Overcoming such technical problems is particularly challenging for onboarding newcomers”; “do not understand what CI does and what it doesn’t”; Seção III.A: “I didn’t know about the linter (or that CI was being used) until after I submitted my patch.”

↓ **reduz** Manutenção do ambiente de desenvolvimento (fator −)
Soltren e Mulligan (2023) *— Enabling Pre-Merge CI on your TV*
Mecanismo: o CI pré-merge obrigatório barra a mudança que quebraria o build, então os builds quebrados caem e o desenvolvimento deixa de parar para desfazer mudanças e resolver conflitos à mão. **Condições:** CI pré-merge tornado obrigatório e somado ao pós-merge, com camada de emulação para acelerar a verificação; base de código grande e multiplataforma. **Evidência:** empírica indireta.
corpo (situação antes, só com verificação pós-submit): “development would sometimes come to a standstill as we backed out changes or resolved conflicts manually”; corpo: “the frequency of build breakages was reduced from 8.70 percent to 4.85 percent in a period of three months”

↑ **aumenta** Percepção de recompensa (fator emergente, Organizacional)
Riedl et al. (2025) *— How an Agile Software Process Increases Developers’ Job Satisfaction: A Stress Perspective Based on the Effort-Reward-Imbalance Model*
Mecanismo: segundo os autores, a CI, junto das demais práticas técnicas, torna a razão esforço-recompensa mais favorável porque aumenta a recompensa percebida (estima, reconhecimento, segurança, carreira), e não porque reduz o esforço. O artigo não detalha como a CI em particular gera essa recompensa. **Condições:** o efeito da CI sobre o estresse é marginal (p = 0,0553); só o efeito indireto sobre a satisfação é significativo. A redução por recompensa foi testada apenas para a agilidade agregada (Tabela 7). Na agilidade agregada, o benefício diminui com o sobrecomprometimento e desaparece no nível alto; incluir o moderador não muda os resultados por prática. Amostra de times que se declaram ágeis. **Evidência:** inferida.
Seção 5, Tabela 6: “the statistics show that requirements change and customer relation have a stress-increasing effect, while the other six dimensions have a stress-reducing effect”; Seção 5, Tabela 7: “agility affects stress predominantly via the reward component”; Seção 6.1: “continuous integration, refactoring, unit testing, collective ownership, and coding standards are agile practices that increase developers’ reward perceptions”

↑ **aumenta** Pressão por manter o build verde (fator emergente, Tarefa e carga de trabalho)
Pinto et al. (2017) *— Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users*
Mecanismo: a norma de que o ramo principal tem de compilar sempre impõe disciplina e auto-organização constantes, vividas como trabalhar sob pressão. **Evidência:** empírica direta.
Seção IV.B: “techniques require developers a certain level of discipline”; “it takes a lot of self-organization to work under pressure of ‘master must build’”

Bots de desenvolvimento no pipeline de CI/CD

5 elos em 1 artigo(s)

↑ **aumenta** Carga cognitiva (fator −)
Asthana et al. (2023) *— A Case Study of Developer Bots: Motivations, Perceptions, and Challenges*
Mecanismo: vários bots comentando no mesmo pull request, com relatórios longos e estatísticas sem explicação, poluem a tela de revisão e obrigam o desenvolvedor a filtrar e priorizar o que importa; o que não se entende acaba ignorado. **Condições:** muitos bots simultâneos; recomendações extensas, sem próximo passo claro ou sem relação com a mudança; bots que indicam a ação exata (B3) não têm esse custo. **Evidência:** empírica direta.
Seção 6.2, D1: “This cluttered the pull-request review screen that developers use to review comments left by peer developers during code review (P18)”; Seção 6.2, D3: “When I see these recommendations, I see a lot of statistics and crash reports. I am not able to connect the dots as to how this would be useful for me.”; Seção 6.2, D2 (P21): “It is not immediately clear to me what I should do with that recommendation, and hence, I mostly ignore it.”; Seção 6.2: “observe that too many bots can cause noise, even if individual bots are helpful”

↕ **ambivalente** Carga de trabalho (fator −)
Asthana et al. (2023) *— A Case Study of Developer Bots: Motivations, Perceptions, and Challenges*
Mecanismo: o bot que pega código esquecido evita a falha de build e o pull request de correção, e o que aponta miudezas tira esse trabalho do revisor; já os bots de qualidade de código acrescentam trabalho ao PR, e o bot que escolhe revisores distribuiu a revisão de forma desigual. **Condições:** reduz com bots que lembram configurações, dependências e regras rotineiras (Nitpicker, Static analysis, ConfigAlert); aumenta com bots de qualidade de código, que pedem melhorias além do que o PR exigia, e com o bot de revisores (achado só citado na discussão, sem dado no texto). **Evidência:** empírica direta.
Seção 6.1, B1 (P16, P17): “increasing their workload. Developers found bot recommendations helpful for "preventing a failure from a missed code" (P17) or "saving on pull-request iterations" (P16)”; Seção 6.1, B4: “Bots take the burden off the code-reviewers to nitpick every minor issue in the pull-request, letting code-reviewers provide feedback on the more important aspects of the review”; Seção 5.2, M4: “The recommendations from these bots may sometimes lead to more turnaround time for the pull-request due to the additional work needed to improve code quality”; Seção 8.2: “unbalanced workload allocation by the "Reviewer" bot”

↑ **aumenta** Conflito de papéis (fator −)
Asthana et al. (2023) *— A Case Study of Developer Bots: Motivations, Perceptions, and Challenges*
Mecanismo: os bots de produtividade e de qualidade cobram boas práticas e desempenho no pull request, o que concorre com a cobrança por terminar o PR no prazo. **Condições:** bots das categorias DevProductivity e CodeQuality; síntese dos autores sobre os desafios relatados pelos usuários. **Evidência:** inferida.
Seção 6.2: “mismatch in priorities between bot developers and bot users”; “However, these may sometimes”; “the pull-request on a deadline”

↑ **aumenta** Documentação interna (fator +)
Asthana et al. (2023) *— A Case Study of Developer Bots: Motivations, Perceptions, and Challenges*
Mecanismo: o bot da equipe de privacidade responde às perguntas frequentes no próprio work item, e o desenvolvedor não fica um dia parado esperando um especialista; o especialista deixa de ser acionado para as dúvidas repetidas. **Condições:** dúvidas rotineiras e repetidas (FAQ) de um time especialista escasso; as questões não triviais continuam com o especialista humano. **Evidência:** empírica direta.
Seção 6.1, B2 (P18): “quick help on basic privacy questions compared to waiting an entire day when someone had to manually answer them”; Seção 5.2, M3 (P11): “has been helping us address security concerns at scale as the bot”; “several routine tasks”

↑ **aumenta** Integração de novatos (fator +)
Asthana et al. (2023) *— A Case Study of Developer Bots: Motivations, Perceptions, and Challenges*
Mecanismo: o bot de configuração lembra o novato das regras e convenções do repositório no próprio pull request, e ele aprende as nuances sem precisar que um sênior aponte cada uma. **Condições:** bots de violação de configuração/estilo (Nitpicker) configurados pelo time para as convenções do repositório. **Evidência:** empírica direta.
Seção 6.1, B1 (P17): “it helps newcomers learn the nuances of”; “without burdening a senior developer to point that out”; Seção 5.2, M1 (P3): “violation bot to guide newcomers towards standard team practices”

Releases frequentes

5 elos em 4 artigo(s)

↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)
Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*
Mecanismo: quanto mais tempo desde a última implantação em produção, menos confiança o desenvolvedor tem na próxima; implantar pouco aumenta o receio de que a próxima falhe e eleva a barreira para implantar, corroendo ainda mais a confiança. **Condições:** contraste entre projetos: no projeto B, que implantava só quando necessário, o receio aumentou; nos de implantação mais frequente não foi relatado. **Evidência:** empírica direta.
Seção 4.2 (projeto B): “Infrequent deployments in project B also increased worries”; “This might also raise the bar for deployments that are not strictly necessary, which would further erode the trust.”; Seção 6: “the less confident the developers feel to make the next production deployment.”

↓ **reduz** Carga cognitiva (fator −)
Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*
Mecanismo: com implantações raras, fica difícil lembrar em que a versão em produção difere da versão em desenvolvimento, e convivem várias versões da mesma funcionalidade; implantar com frequência mantém a distância entre as duas versões pequena. **Condições:** relatado pelo entrevistado do projeto C, onde o cliente pedia que a implantação fosse feita do escritório dele e a média era de duas implantações por mês. **Evidência:** empírica direta.
Seção 4.2 (projeto C): “the interviewee in project C mentioned”; “When the deployments were only made on particular necessity, the deployments were not always that frequent in both projects B and C.”; “how the version in production was different from the current development version.”

↓ **reduz** Conflito trabalho-família (fator −)
Claes et al. (2017) *— Abnormal Working Hours: Effect of Rapid Releases and Implications to Work Content*
Mecanismo: com a passagem para um ciclo de release a cada 6 semanas, a fatia da atividade de desenvolvimento feita em fins de semana e à noite caiu, embora o volume em horário comercial tenha crescido; os autores contrariam a hipótese de que prazos mais frequentes aumentariam a pressão e empurrariam o trabalho para fora do expediente. **Condições:** Mozilla Firefox, mudança de releases anuais com versões paralelas para releases a cada 6 semanas em 2011; queda maior nos fins de semana (d = 1.43) que à noite (d = 0.45); considerando só endereços Mozilla, fins de semana d = 0.64 e noite com efeito pequeno a desprezível; parte da diferença pode vir do aumento de desenvolvedores pagos pela Mozilla; trabalho fora de hora medido pelo horário de comentários de patch e revisão; uma só organização. **Evidência:** empírica direta.
Seção IV.B: “There is a statistically significant decrease in the relative amount of comments made outside office hours.”; Seção IV.B: “On average since the release of Firefox 4, a weekend day receives 24.4% of the comments a weekday receives, while before Firefox 4, a weekend day used to receive 48.6% of the comments of a weekday.”; Seção IV.B: “we still observe a large decrease in the activity during weekends”; Seção VI: “switching to rapid releases has reduced the amount of work performed outside office hours”

↓ **reduz** Pressão de prazos (fator −)
Rossi et al. (2016) *— Continuous deployment of mobile software at facebook (showcase)*
Mecanismo: com corte do release branch semanal, perder o corte custa só uma semana de espera, então o desenvolvedor não força o push no dia do corte e a corrida contra o prazo diminui. **Condições:** release branch cortado em data fixa; com ciclo mais longo (duas semanas, iOS) a proporção de pushes no dia do corte é maior; mover o corte para o fim de semana reduz ainda mais a corrida, mas aumentou os pushes no fim de semana. **Evidência:** empírica indireta.
Seção 5.4 Deadline Effects: “There is a clear increase in the number of pushes to the Master branch on the day the Release branch is cut. This may indicate that developers rush to get their code in before the deadline”; Seção 5.4: “if a developer believes her update will not make the cut, they will only have to wait one week and hence do not force a push”; Seção 5.4: “there is a lower tendency to rush to push updates when the cut date is on a weekend”; Seção 7 Concluding Remarks: “Shortening the release cycle improved this, because developers were less likely to rush their software pushes”

↑ **aumenta** Pressão de prazos (fator −)
Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*
Mecanismo: a cadência rápida de entregas impõe datas de entrega sem tempo para fazer o trabalho direito; etapas como testes são encurtadas para cumprir o prazo. **Condições:** fator comum aos modos ágil e DevOps, não específico de DevOps; os entrevistados pedem períodos de descanso entre um prazo e outro. **Evidência:** empírica direta.
Findings, Dissatisfaction, fator 2 (PM-DO): “Deployments too fast: it’s sometimes frustrating to have delivery dates imposed on us without having properly or correctly done our jobs.”; Findings, Risks, fator 2: “Increased work cadences, resulting in work overload and tasks prioritization”; Discussion: “Indeed, some steps such as test phases can be lightened to meet deadlines”; “According to the interviewees, it seems important to provide rest periods between each deadline.”

Site Reliability Engineering

5 elos em 1 artigo(s)

↑ **aumenta** Alinhamento do uso do tempo (fator +)

Kafoe et al. (2026) — *"making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil*

Mecanismo: automatizar o toil libera o tempo antes gasto em tarefa manual e o devolve ao trabalho de engenharia e a iniciativas novas, que é o que tira o engenheiro da 'engenharia de verdade'. **Condições:** o custo de manter a automação, incluindo documentação, costuma ser subestimado e consome parte do ganho; eliminação exige retorno de investimento alto para se justificar. **Evidência:** empírica direta.

4.3.2 (CS-1, sobre o toil): “[toil] keeps you from the actual engineering.”; 4.4.2: “elimination increases teams’ velocity (cf., reduces teams’ velocity) by freeing up time for engineering tasks”; 4.4.2: “as engineers can redirect saved time toward creative problem-solving and new initiatives”

↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)

Kafoe et al. (2026) — *"making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil*

Mecanismo: eliminar o toil tira do dia a tarefa repetitiva e monótona e o ato de automatizar exige pensar fora da caixa, o que dá mais variedade e criatividade ao trabalho. **Evidência:** empírica direta.

4.4.2 (EJ-2): “helps in making our life less monotonous and more flexible for something else.”; 4.4.2: “forces creativity since eliminating toil demands thinking outside of the box as the task is far from trivial”; 4.2.2 (CM-1, sobre o toil): “[D]oing the same thing every day and you’re sort of like in a box.”

↕ **ambivalente** Desenvolvimento de competências (fator emergente, Individual)

Kafoe et al. (2026) — *"making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil*

Mecanismo: o tempo liberado e o desafio de automatizar favorecem o desenvolvimento pessoal, mas a automação tira a prática direta com o sistema, reduz a transparência e corrói o conhecimento do sistema e a habilidade de depuração. **Condições:** o ganho de desenvolvimento vem do tempo redirecionado para engenharia; a perda vem de a automação esconder o funcionamento que antes se aprendia fazendo à mão. **Evidência:** empírica direta.

4.4.2: “participants agree that toil elimination increases morale (cp., reduces morale) and improves personal development”; 4.4.2: “hinders system comprehension as related automation reduces transparency and erodes the system knowledge”; 4.4.2: “Toil elimination also impacts debugging skills as engineers lose hands-on experience working directly with the system”

↑ **aumenta** Insegurança no emprego (fator −)

Kafoe et al. (2026) — *"making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil*

Mecanismo: quem tem o trabalho centrado em toil passa a se sentir substituível quando a tarefa é automatizada. **Condições:** engenheiros alocados predominantemente em toil; os autores sugerem não alocar ninguém exclusivamente em toil para evitar turnover e resistência à eliminação. **Evidência:** empírica direta.

4.4.2: “threaten [the engineers’] job”; 4.4.2: “engineers working on toil might feel replaceable”; quadro RQ3: “especially among employees dedicated to toil, who may feel threatened by automation”

↕ **ambivalente** Senso de realização (fator +)

Kafoe et al. (2026) — *"making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil*

Mecanismo: construir a automação que elimina o toil dá a satisfação de resolver o problema; mas eliminar o toil também tira a tarefa rápida e conhecida que permite 'riscar coisas da lista' e sentir que o dia rendeu. **Condições:** aumenta para quem constrói a solução de eliminação; reduz para quem se sente valorizado ao fazer toil ou não gosta do processo de eliminação. **Evidência:** empírica direta.

4.4.2 (CM-2): “it’s obviously the satisfaction [that] comes from eliminating [toil] and building solutions”; 4.4.2: “toil elimination reduces morale given that some engineers feel valued when performing toil”; 4.4.2 (CM-3): “[y]ou can do [toil] quickly and without a lot of thought, and you can just tick things off, and sometimes that really helps with how you perceived your day to have gone”

Automação de testes

4 elos em 4 artigo(s)

↑ **aumenta** Fadiga de alertas (fator emergente, Tarefa e carga de trabalho)
Andreyev (2025) — *The Art of Managing Flaky Tests at Scale*
Mecanismo: cada teste flaky que passa do limiar vira item de ação com prazo de um dia útil; com limiar agressivo, os alertas excedem a capacidade dos times para dívida técnica, eles se dessensibilizam e passam a desabilitar o teste em vez de corrigi-lo. **Condições:** limiares de falha agressivos; ajustar o limiar à capacidade dos times atenuou o efeito (correções na primeira investigação quase dobraram); limiar frouxo demais deixa o teste instável no pipeline. **Evidência:** empírica indireta.
Página 1: “Action items must be addressed within one business day, as all required tests must pass before a change can be merged.”; “overly aggressive alerting thresholds led to a counterproductive "resolve now, investigate later" mentality”; “alert fatigue significantly impacts long-term process adherence”; “Teams have a finite capacity for addressing technical debt, and excessive alerts can lead to desensitization”; “teams in aggregate nearly doubled - from 23.9% to 44.8% - by fine-tuning flaky test thresholds”

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)
Kärpänoja et al. (2016) — *Exploring peopleware in continuous delivery*
Mecanismo: testes end-to-end automatizados em navegador real demoram mais de uma hora e falham às vezes sem motivo lógico, pela instabilidade dos frameworks de teste, e consomem tempo extra para torná-los confiáveis. **Condições:** específico de testes end-to-end em navegador real (projetos A e C); um time passou a escrever testes de navegador só para a funcionalidade central, como forma de conter o custo. **Evidência:** empírica direta.
Seção 4.1: “Especially automated end-to-end tests that are run with real browsers were mentioned as being a pain for the”; “Though important, such tests may take more than an hour to run and may fail occasionally without any logical reason because of the unreliability of the test frameworks being used. Often some extra time was spent to improve tests and achieve better reliability.”; “they stopped writing new browser tests for other than just core functionality at one point.”

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)
Parry et al. (2022) — *Surveying the Developer Experience of Flaky Tests*
Mecanismo: testes automatizados que passam e falham sem mudança no código geram falhas falsas que custam produtividade: o desenvolvedor reexecuta o build, tenta reparar um teste cuja causa muitas vezes não se descobre e acaba sem tempo para consertar todos. **Condições:** depende da frequência com que a suíte apresenta testes instáveis: quem os vê ao menos mensalmente reexecuta mais, tende mais a não agir e repara menos; causas mais citadas são setup/teardown inadequado, dependência de rede e causas desconhecidas. **Evidência:** empírica direta.
RQ2, conclusão (SQ4): “They also agree that flaky tests lead to both a loss of productivity and a reduction in testing efficiency.”; RQ4 (SQ7): “The most common action as scored by all participants was to simply re-run the failing build (SQ7.2)”; RQ4 (SQ7.1): “This suggests that developers who experience flaky tests more often are more likely to take no action.”; Tabela 1, SQ11t8 (P64): “We haven’t got the time to address them all.”

↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)
Ahmad et al. (2023) — *An Industrial Study on the Challenges and Effects of Diversity-Based Testing in Continuous Integration*
Mecanismo: a suíte de regressão automatizada, executada ao menos uma vez por dia, acumula testes antigos ou desatualizados que falham quando as regras de escrita mudam ou quando a rede do ambiente de teste falha, e a checagem e atualização é feita à mão, um fardo para a equipe. **Condições:** produto configurável com muitas combinações de features; diretrizes de escrita de teste que mudam ao longo do tempo; pouca documentação de processo; ambiente de teste com rede instável; checagem manual das inconsistências; relato de 4 testadores seniores de uma empresa. **Evidência:** empírica direta.
Seção 4.2, Tema 1: “Identifying and handling old or outdated test cases (C3) is also one of their challenges with continuous testing”; P1: “It’s a burden on us.”; Tema 2 (P3): “sometimes all the test cases fail and we look at them and we see for obvious reason that this is a very good test case written three years ago and they changed the rules during the time”; Tema 3 (P1): “Most of the time when we have flaky test cases, it is because of the test environment, the network really not working as it should.”; Seção 5: “Currently, these checks are done manually when issues during execution arrive (e.g., failures), which is detrimental to stability of test cycles (SI2).”

Times multifuncionais Dev–Ops

4 elos em 1 artigo(s)

↓ **reduz** Apoio do gestor (fator +)

Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*

Mecanismo: o time DevOps reúne gente de várias linhas e funciona à margem das linhas hierárquicas; os chefes de cada linha perdem o controle do time e deixam de dar retorno e atenção. **Condições:** empresa com duas linhas hierárquicas separadas (Dev e Ops) e times remotos; nas comparações com outras empresas, com linha hierárquica unificada e painel comum, o feedback da hierarquia era bem maior. **Evidência:** empírica indireta.

Findings, Dissatisfaction, fator 6: “DevOps teams are composed of members of various trade and has an own operating mode that is independent of the hierarchical lines and traditional craft.”; (DM-DO): “My hierarchy is not very present, so I don’t have too much feedback from my boss.”; Dissatisfaction and risks: “Since the DevOps project team is multi-line of services, the superiors have no control over the team and may seem overwhelmed by this cross-functional mode of operation”; Conclusion: “Comparisons we made with other firms showed that when there is a unified hierarchical line with a common dashboard, feedback from the hierarchy was much more important.”

↑ **aumenta** Conflito de papéis (fator −)

Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*

Mecanismo: o operador integrado ao projeto DevOps continua responsável pela operação e pelos incidentes; tem de tocar projeto e incidente em paralelo e o incidente atrapalha as tarefas em que queria se concentrar. **Condições:** operadores que mantêm suporte de nível 1 em outras aplicações; os autores ligam o efeito à alta disponibilidade diária exigida pelo modo DevOps. **Evidência:** empírica direta.

Findings, Dissatisfaction, fator 5: “Some production people also complain about the overload of work in their department, since they are responsible for DevOps projects and incident management to be conducted in parallel.”; (PE-DO): “We have applications on which we are still operating level 1. We do a lot of things at once, there are tasks on which we would like to focus ... sometimes we are a little disturbed by the incident.”; Dissatisfaction and risks: “The DevOps approach requires a very high level of availability and involvement on a daily basis, which makes it difficult for the operators to manage projects and incidents concomitantly”

↕ **ambivalente** Reconhecimento no trabalho (fator +)

Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*

Mecanismo: o time misto dá visibilidade ao trabalho de Ops, que passa a ser reconhecido pelos demais; ao mesmo tempo, os papéis novos e transversais de PO e RM ficam expostos e precisam justificar suas escolhas de método diante dos outros. **Condições:** aumenta para engenheiros de produção e arquitetos, cujo papel ganha lugar no time; reduz para PO e RM, papéis transversais e sem linha hierárquica, julgados pela abordagem experimental. **Evidência:** empírica direta.

Findings, Satisfaction, fator 4 (PE-DO): “We are more recognized than before”; Findings, Dissatisfaction, fator 4: “They sometimes feel judged by others in their innovative project management approach, feeling that they have to justify their choices.”; (PO-DO): “Sometimes I feel that I have to justify our choices, and finally the methodology, the impression that we are judged in fact.”; Dissatisfaction and risks: “This factor of dissatisfaction hinders the overall satisfaction felt by a majority of team members about the role to play and its recognition.”

↑ **aumenta** Relações no trabalho (fator +)

Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*

Mecanismo: a entrada de Ops no time de projeto multiplica as trocas e as relações entre papéis e une o time; o engenheiro de produção passa de executor a parceiro. **Condições:** efeito mais forte no modo DevOps que no ágil: nos times ágeis, com o silo Dev/Ops ainda de pé, Ops relatam expectativa frustrada de colaboração com Dev; a dispersão geográfica do time é desfavorável, e mais ainda em DevOps. **Evidência:** empírica direta.

Findings, Satisfaction, fator 4 (PE-DO): “We are more recognized than before and now there are more exchanges, there are more relationships; we went from being a performer to a partner position.”; Findings, Satisfaction, fator 9: “Through the DevOps approach, the teams seem unified”; Table 2: “DevOps practices: group dynamics and the collective ability to collaborate”; Specific Dissatisfaction for Agile Teams (RM-AG): “they do not listen to us, because the voice of the run is not important, whereas it should be!”; Discussion: “this dispersion appears as unfavorable (although not an impossible barrier to overcome) to agile and even more to DevOps practices”

Gestão de incidentes

3 elos em 1 artigo(s)

↑ **aumenta** Carga cognitiva (fator −)
Shetty et al. (2022) *— AutoTSG: learning and synthesis for incident troubleshooting*
Mecanismo: o runbook manual não garante a solução, e o plantonista precisa julgar, incidente a incidente, quais passos se aplicam. **Condições:** serviço muito customizado; TSG manual (não executável). **Evidência:** empírica direta.
Seção 4.4, Q2 (P6): “I work on a service with a lot of customization. So when incidents arise, I have to manually verify which steps in a TSG will work for that incident. There is no guaranteed solution.”

↓ **reduz** Carga de trabalho (fator −)
Shetty et al. (2022) *— AutoTSG: learning and synthesis for incident troubleshooting*
Mecanismo: o runbook (TSG) ligado ao incidente encurta o tempo de mitigação dos incidentes que acionam o plantonista, e com isso o esforço de plantão gasto por incidente. **Condições:** incidentes de severidade 1 e 2 (com paging); TTM usado pelos autores como proxy do esforço de plantão. **Evidência:** empírica indireta.
Seção 2.1 (TSGs and Time-to-mitigate): “TTM, is a reliable proxy for on-call effort”; “We find that the mean TTM of incidents without TSGs (≈19 hrs) is significantly higher than that of incidents linked to a TSG (≈13 hrs).”; “TSGs tend to significantly reduce effort during incident mitigation”

↑ **aumenta** Interrupções no trabalho (fator −)
Shetty et al. (2022) *— AutoTSG: learning and synthesis for incident troubleshooting*
Mecanismo: quando o runbook é ruim, confuso ou desatualizado, quem está de plantão recorre aos colegas, e quem conhece o sistema passa a ajudar os pares todos os dias, fora da própria escala. **Condições:** TSGs de baixa qualidade (incompletos, desatualizados, confusos). **Evidência:** empírica direta.
Seção 4.4, Q1 (P2): “My usual on-call rotation is bi-weekly. But I end up helping my peers use TSGs every day, because they are bad and confusing.”; Seção 4.4, Q2 (P7): “Mostly outdated TSGs or missing the information I need. So I generally discuss with my team for many of the issues.”

Responsabilidade de operação pelo time

3 elos em 2 artigo(s)

↑ **aumenta** Conflito trabalho-família (fator −)
Jeyam et al. (2026) *— "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI*
Mecanismo: o próprio time DevOps faz as implantações das releases mensais e dos lançamentos urgentes, que são escaladas para a noite e o fim de semana, conforme quem estiver em melhor posição. **Condições:** releases mensais ou urgentes que podem ir para produção a qualquer hora; o artigo cita responsabilidade compartilhada (Duty Days) como coping. **Evidência:** empírica direta.
Seção 4.2: “Monthly releases or urgent launches often required work to be done in the evenings or on the weekends.”; Seção 4.2 (Agile Project Manager, Nr. 22): “So who is deploying right now, on the weekend, for example. Then you just see who is in the best position to do it”; Seção 4.2 (SE (Dev), Nr. 12): “Theoretically, we could release it at any time during the night or at some point over the weekend”

↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)
Kärpänoja et al. (2016) *— Exploring peopleware in continuous delivery*
Mecanismo: deixar o time cuidar da infraestrutura, das configurações de servidor e da entrega, além de testes e qualidade, dá mais responsabilidade e controle sobre o sistema e exige um repertório mais amplo de competências; onde o acesso a componentes e servidores era negado, os desenvolvedores se sentiam menos responsáveis pelos problemas ali. **Condições:** projetos com configuração automatizada (IaC) em dois dos três casos; para um entrevistado, a responsabilidade maior também aumentou o estresse, pela exigência de competência mais ampla; o efeito depende de o time ter acesso real aos componentes. **Evidência:** empírica indireta.
Seção 4.1: “letting them manage the infrastructure gives them more responsibility and control over the system.”; Seção 4.3: “one interviewee also pointed out that it increased stress.”; “When the development team is also responsible for server configurations, testing and quality, wider level of”; “they sometimes felt careless if problems were related to these components.”; “the development team more access and responsibility seems to improve motivation and commitment to the project”

↑ **aumenta** Pressão por atualização (fator −)
Jeyam et al. (2026) *— "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI*
Mecanismo: assumir infraestrutura e segurança além do desenvolvimento amplia o leque de competências exigidas, e para acompanhar as pessoas passam a estudar e programar no tempo livre. **Condições:** atenuado quando a organização reserva tempo de treino, oferece cursos internos e workshops e o time compartilha conhecimento (pair programming); sem tempo alocado, o aprendizado fica para fora do horário. **Evidência:** empírica direta.
Seção 4.2 (SE (Dev), Nr. 5): “We do DevOps, which means we do infrastructure, we have to take care of security, there are requirements coming from everywhere that you have to meet somewhere”; Seção 4.2: “their skills during personal times in order to keep up with the broad expertise which is required from DevOps members”; Seção 4.2 (SE (Dev), Nr. 16): “I did a lot of things in my free time. That’s when I started doing a bit of programming and trying things out, learning new things”; Seção 4.3 (Application Manager, Nr. 24): “the biggest challenge is taking the time to do it”

Implantação contínua

2 elos em 2 artigo(s)

↑ **aumenta** Carga cognitiva (fator −)
Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*
Mecanismo: com a automação das releases, o engenheiro de produção deixa de saber o que mudou na aplicação; defeitos funcionais ficam escondidos e a manutenção posterior fica mais difícil de entender. **Condições:** risco percebido por engenheiros de produção, que mantêm a aplicação depois; o artigo fala só em automação (de releases, pela definição de maturidade DevOps), sem separar CI e deploy; agravado quando o investimento em tempo e em automação é insuficiente. **Evidência:** empírica direta.
Findings, Risks, fator 6: “Individuals evoke risks that may remain hidden by automation and make more complex the application maintenance.”; (PE-DO): “With automation, we have less and less knowledge about the app. Today with automation, I don’t know what has changed and I have no vision of what has been done.”; Dissatisfaction and risks: “automation can hide some functional defects in the developed application, which PE’s will have difficulty maintaining later”

↓ **reduz** Carga de trabalho (fator −)
Zhang et al. (2018) *— One size does not fit all: An empirical study of containerized continuous deployment workflows*
Mecanismo: o workflow de CD constrói e publica a imagem automaticamente a cada mudança, eliminando os passos manuais de build e deploy que antes cabiam ao desenvolvedor. **Evidência:** empírica direta.
Seção 3.2, Tabela 1: “Helps us deploy automatically instead of doing it manually 60 42.9%”; Seção 3.2 [M1] (R79): “because the project will be built automatically, no need for me to build the image and push it to the server”

Postmortem sem culpados

2 elos em 1 artigo(s)

↕ **ambivalente** Carga de trabalho (fator −)
Anandayuvaraj et al. (2026) *— Learning from Software Failures: A Case Study at a National Space Research Center*
Mecanismo: a reunião formal de reflexão sobre falhas soma mais uma reunião e mais burocracia a uma agenda já tomada por projeto, mas evita o retrabalho de resolver de novo as mesmas falhas recorrentes. **Condições:** aumenta quando vira reunião formal e frequente (mensal ou quinzenal) num ambiente já burocrático; reduz quando é leve e encaixada no processo existente, p. ex. nos marcos do projeto; efeitos antecipados pelos participantes, pois a prática não estava adotada na organização. **Evidência:** empírica indireta.
§5.3.1 (S3): “we have all the bureaucracy, every additional meeting puts a big hole in your calendar”; §5.3.1 (S8): “if it’s something like twice a month or even once a month, for me it will become a little bit tedious.”; §5.3.1 (S1): “That’s a bad excuse not having the time when we save time by this.”; §5.3.1 (S5): “It would be helpful to have a process to learn from mistakes...because it takes a lot of time if people make the same mistakes over and over again.”; §5.3.1 (S9): “if you do it [with] milestones...then it might be a huge benefit.”

↑ **aumenta** Documentação interna (fator +)
Anandayuvaraj et al. (2026) *— Learning from Software Failures: A Case Study at a National Space Research Center*
Mecanismo: sem um processo de postmortem que exija registrar as lições, o conhecimento sobre falhas fica tácito, na cabeça de quem resolveu, e se perde quando a pessoa sai; o registro, quando existe, poupa tempo de quem chega. **Condições:** evidência pela ausência da prática: os participantes ligam a falta de registro à inexistência do processo, sobretudo em times com rotatividade alta. **Evidência:** empírica indireta.
§5.1.2 (S4): “when we close the issue, it is not mandatory to document what did you do...[it’s a] personal decision. We don’t have that process”; §5.3.3 (S2): “[Lessons] should be explicitly documented because experience is just related to one person”; “losing the lesson learned...because there’s no process to say ‘part of your project is to write lesson learned’.”; §5.3.3 (S4): “I have used what other team members have written [about issues]...it saved me a lot of time”

ESC/DevOps em geral

1 elo em 1 artigo(s)

↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)
Hemon et al. (2018) *— Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams*
Mecanismo: o escopo mais amplo do trabalho em DevOps faz a pessoa tocar muitas áreas, usar ferramentas e tecnologias variadas e aprender todo dia, o que aumenta a variedade de tarefas. **Condições:** a variedade de tarefas é fator comum aos modos ágil e DevOps; que ela seja maior em DevOps é argumento dos autores, pelo escopo mais amplo, sem dado comparativo. **Evidência:** empírica indireta.
Findings, Satisfaction, fator 1: “DevOps methodology has satisfied several interviewed individuals, who appreciate testing varied teamwork methods, several tools and technologies, or benefiting from a daily learning mode in their profession.”; (RM-DO): “We still touch a lot of areas, so it's an interesting job for that.”; Discussion: “With a wider scope of activity, it can be argued that sequential variety of DevOps team is also larger than that of agile teams.”

Monitoramento e observabilidade

1 elo em 1 artigo(s)

↕ **ambivalente** Carga cognitiva (fator −)
Jeyam et al. (2026) *— "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI*
Mecanismo: acompanhar incidentes por várias caixas de e-mail, sistema de tickets e dados de monitoramento divide a atenção e sobrecarrega; alertas automáticos para uma caixa do time, com os dias repartidos entre as pessoas, tiram o peso da vigilância constante. **Condições:** aumenta quando o monitoramento é espalhado por vários canais e gera volume de dados; reduz quando há alerta automático por exceção e divisão dos dias de atendimento no time. **Evidência:** empírica indireta.
Seção 4.2 (SE (Dev), Nr. 12): “You have to monitor everything that goes wrong, process multiple mailboxes, and operate the ticket system.”; Seção 4.2: “This constant multitasking, while necessary for ensuring platform stability, can disrupt focus and contribute to mental fatigue.”; Seção 4.2: “had become a source of overwhelming data, especially outside of critical incidents”; Seção 4.3 (SE (Dev), Nr. 5): “With the latest applications, we have really set up monitoring so that if certain exceptions occur, we are notified by email. As a team, we have an application mailbox where everything is sent, and we have divided up the days accordingly.”; Seção 4.3: “These systems reduce the mental burden of constant vigilance”

On-call

1 elo em 1 artigo(s)

↑ **aumenta** Conflito trabalho-família (fator −)

Jeyam et al. (2026) *— "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI*

Mecanismo: o plantão 24/7 obriga a ficar disponível o tempo todo e, quando algo quebra à noite, quem está de plantão liga para o colega especialista, de modo que mesmo fora da escala a pessoa se sente sempre ligada. **Condições:** plantão 24/7 com dependência de especialistas; o artigo apresenta a responsabilidade compartilhada em rodízio (Duty Days, escalas divididas) como coping, mas a fala correspondente trata de repartir pedidos, não de horário. **Evidência:** empírica direta.

Seção 4.2: “on-call or standby duties further contribute to this sense of invasion”; Seção 4.2 (SE (DevOps), Nr. 23): “Because we also have standby duty, 24/7, we are very dependent on each other. So it may be that a developer is on standby duty, and if something comes up on the platform at night, it’s normal that they call their colleague who is the specialist in that area”; Seção 4.2: “This constant availability created a sense of being always”; Seção 4.3 (SE (Dev), Nr. 7): “so that not everyone has to constantly process requests”

5. Fichas dos artigos

O que cada artigo estuda e conclui segundo o texto completo, e os elos que sustenta.

Com elo prática → fator (22)

Anandayuvaraj et al. (2026) — Learning from Software Failures: A Case Study at a National Space Research Center

Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering, ICSE 2026

Estudo de caso qualitativo (ICSE 2026) com 10 entrevistas com engenheiros de software de pesquisa num centro espacial nacional, mais 5 entrevistas piloto em outras organizações de alta confiabilidade, sobre como se aprende com falhas (postmortem/retrospectiva). O achado central é a ausência de processo estruturado de postmortem: o aprendizado é informal, as falhas se repetem e o conhecimento se perde com a rotatividade e a documentação fragmentada. O elo com as pessoas é fraco e vem sobretudo de percepções sobre a prática ainda não adotada: os participantes antecipam reuniões de reflexão como burocracia que consome tempo, mas também dizem que ela pouparia o retrabalho de falhas recorrentes; e atribuem a falta de registro das lições à inexistência do processo. O contexto não é de DevOps/entrega contínua, e o lado sem culpados do postmortem não aparece nos dados.

Postmortem sem culpados ↕ **ambivalente** Carga de trabalho (fator −); Postmortem sem culpados ↑ **aumenta** Documentação interna (fator +)

Jeyam et al. (2026) — "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI

Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP 2026

Estudo qualitativo com 26 entrevistas semiestruturadas com membros de times DevOps de duas empresas suíças reguladas (seguradora e banco), analisadas pelo modelo de tecnoestresse de Tarafdar (complexidade, sobrecarga, invasão, insegurança, incerteza) e por estratégias de coping em quatro níveis. As falas ligam plantão 24/7 e implantações noturnas e de fim de semana à invasão da vida pessoal, a responsabilidade ampliada do DevOps ao estudo fora do horário, o monitoramento por vários canais à sobrecarga cognitiva e a heterogeneidade dos pipelines de CI/CD à incerteza e à troca de contexto; escala dividida (Duty Days), alertas automáticos com dias repartidos, tempo de treino e padronização de pipelines aparecem como atenuantes. A parte de GenAI (estressor e mitigador) não envolve prática de ESC e não gerou efeito.

On-call ↑ **aumenta** Conflito trabalho-família (fator −); Responsabilidade de operação pelo time ↑ **aumenta** Conflito trabalho-família (fator −); Responsabilidade de operação pelo time ↑ **aumenta** Pressão por atualização (fator −); Monitoramento e observabilidade ↕ **ambivalente** Carga cognitiva (fator −); Pipeline de CI/CD ↕ **ambivalente** Carga cognitiva (fator −)

Kafoe et al. (2026) — "making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil

Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering, ICSE 2026

Estudo exploratório multimétodo (ICSE 2026): revisão de literatura cinzenta (61 fontes, blogs e posts de praticantes) seguida de estudo de caso com 11 entrevistas semiestruturadas (6 engenheiros e 5 coordenadores de operações) na ING, que adotou SRE e faz oficinas de eliminação de toil. O elo com as pessoas vem da eliminação de toil como prática de SRE: os entrevistados relatam trabalho menos monótono, mais tempo para engenharia e desenvolvimento pessoal, mas também perda da 'vitória rápida' que dá sensação de dia produtivo, perda de conhecimento do sistema e de habilidade de depuração, e ameaça ao emprego de quem vive de toil. Burnout, moral e turnover aparecem como desfechos (não como fatores), sem teste quantitativo; a ligação toil → facilita onboarding (CS-1) também aparece, mas o lado da eliminação só vem da literatura cinzenta e ficou fora dos efeitos.

Site Reliability Engineering ↑ **aumenta** Desenho do trabalho (fator −, nome neutro*); Site Reliability Engineering ↑ **aumenta** Alinhamento do uso do tempo (fator +); Site Reliability Engineering ↕ **ambivalente** Senso de realização (fator +); Site Reliability Engineering ↑ **aumenta** Insegurança no emprego (fator −); Site Reliability Engineering ↕ **ambivalente** Desenvolvimento de competências (fator emergente, Individual)

Kassab et al. (2026) — The Emotional Cost of Technical Debt: Quantifying Developer Frustration Across Debt-Related Concern Themes

FSE Companion 2026 - Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering

Mineração de 73.125 comentários de issues sobre dívida técnica em três projetos Apache (Hadoop, Kafka, Flink), com LDA em 7 temas e classificador de emoção (índice NEI). O tema de dívida de testes e infraestrutura de CI/CD (testes flaky, pipelines de build instáveis) tem NEI mais alto que os temas de API e de código, mas o efeito é pequeno (Cliff's δ = 0,06) e os autores não afirmam causalidade; a comparação é entre tipos de dívida, não entre ter ou não CI. O tema de observabilidade e conectividade tem o NEI mais alto, mas mistura configuração, SSL e conexão, e a ligação com a carga de on-call é afirmação dos autores sem dado: não registrado.

Pipeline de CI/CD ↑ **aumenta** *desfecho:* emoção negativa expressa nas discussões (sobretudo tristeza/frustração); Pipeline de CI/CD ↑ **aumenta** Interrupções no trabalho (fator −)

Andreyev (2025) — The Art of Managing Flaky Tests at Scale

2025 IEEE International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2025

Relato de experiência de duas páginas (workshop CCIW no ICSTW 2025, Okta) sobre uma iniciativa de um ano para controlar testes flaky na CI de um monorepo: os alertas de teste flaky exigem ação em um dia útil, e o autor variou os limiares de alerta medindo a taxa de testes corrigidos na primeira investigação (de 23,9% para 44,8%). O único dado é essa taxa; a leitura de que limiares agressivos geram fadiga de alertas e dessensibilização dos times é do autor, a partir dela. Dashboards, calendário de incidentes da CI, tutoriais, office hours, sorteios e shout-outs teriam mudado a visão dos flaky tests de fardo para responsabilidade coletiva, mas isso é afirmação sem dado e as iniciativas não são práticas de ESC; não gera efeito.

Automação de testes ↑ **aumenta** Fadiga de alertas (fator emergente, Tarefa e carga de trabalho)

Riedl et al. (2025) — How an Agile Software Process Increases Developers' Job Satisfaction: A Stress Perspective Based on the Effort-Reward-Imbalance Model

Business and Information Systems Engineering

Survey transversal com 178 desenvolvedores de times ágeis na Áustria, com mediação moderada (PROCESS, modelos 4 e 7): agilidade, medida por oito práticas do instrumento de Maruping et al. (2009), reduz o estresse medido como desequilíbrio esforço-recompensa (ERI) e assim aumenta indiretamente a satisfação; o efeito vem da percepção de recompensa, não do esforço, e some nos muito sobrecomprometidos. Das oito práticas, só integração contínua e teste unitário são de ESC. Para CI, o efeito sobre o estresse fica no limite (β = -0,0151, p = 0,0553, IC até 0,0002), mas o efeito indireto sobre a satisfação é significativo. A ligação de CI à recompensa é inferência dos autores: a decomposição esforço × recompensa só foi feita para a agilidade agregada. Teste unitário não teve efeito significativo sobre o estresse (p = 0,10) nem efeito indireto sobre a satisfação (IC inclui zero), por isso não entrou como efeito.

Integração contínua ↑ **aumenta** Percepção de recompensa (fator emergente, Organizacional)

Fluri et al. (2024) — On the importance of CI/CD practices for database applications

Journal of Software: Evolution and Process

Revisão sinóptica da literatura seguida de três estudos de caso industriais (Oracle; 20, 7 e 35 desenvolvedores) em que os autores implantaram um pipeline automatizado de CI/CD, com métricas 5 semanas antes e 5 depois e 12 entrevistas de 90 min (dois desenvolvedores por caso, antes e depois). As entrevistas só são resumidas pelos autores, sem falas literais (por sigilo), e uma das autoras era consultora das empresas. Os autores relatam que, depois do pipeline, desenvolvedores e coordenadores de release ficaram com menos carga cognitiva, menos interrupções por troubleshooting de integração e implantação, mais confiança ao implantar e menos dependência do coordenador de release. Não se mediu efeito sobre a hora extra relatada antes (implantações fora do expediente por pacote único de migração), e o lead time não melhorou onde a empresa manteve janelas fixas de release.

Pipeline de CI/CD ↓ **reduz** Carga cognitiva (fator −); Pipeline de CI/CD ↓ **reduz** Interrupções no trabalho (fator −); Pipeline de CI/CD ↑ **aumenta** Desenho do trabalho (fator −, nome neutro*); Pipeline de CI/CD ↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)

Valenzuela-Toledo et al. (2024) — The Hidden Costs of Automation: An Empirical Study on GitHub Actions Workflow Maintenance

Proceedings - 2024 IEEE International Conference on Source Code Analysis and Manipulation, SCAM 2024

Estudo de mineração de repositórios (183 projetos maduros do GitHub, 10 linguagens) sobre a manutenção dos arquivos de workflow do GitHub Actions: volume, taxa de mudança, acoplamento lógico com código de produção e teste, motivos do acoplamento (card sorting de 200 commits) e quem faz a manutenção. Conclui que o pipeline automatizado tem um custo escondido: os workflows mudam todo mês, sobretudo para melhorias de CI/CD e correção de bugs, e essa manutenção manual fica concentrada em cerca de 5% dos desenvolvedores, que também mexem em código de produção e teste. Não mede nada nas pessoas diretamente: o esforço é inferido a partir dos commits, e os autores reconhecem que contar mudanças é só um proxy da manutenção (ameaça à validade de construto). Por isso entra um único elo, com evidência indireta.

Pipeline de CI/CD ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Ahmad et al. (2023) — An Industrial Study on the Challenges and Effects of Diversity-Based Testing in Continuous Integration

IEEE International Conference on Software Quality, Reliability and Security, QRS

Estudo de caso na Axis Communications: grupo focal de 3 horas com 4 testadores seniores e análise temática dos desafios do teste de regressão em CI e dos efeitos percebidos da priorização de testes por diversidade (DBT), depois de uma demonstração. Quase tudo trata de adoção da técnica e de eficácia dos testes; o único elo com quem trabalha é o fardo de manter a suíte automatizada executada continuamente (testes desatualizados, falhas por mudança de regra e por ambiente de rede, checagem manual). Os efeitos da DBT (liberar tempo para teste exploratório, transparência) são prospectivos, após a demo, e o de teste exploratório se refere a teste manual: não registrados.

Automação de testes ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Asthana et al. (2023) — A Case Study of Developer Bots: Motivations, Perceptions, and Challenges

ESEC/FSE 2023 - Proceedings of the 31st ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering

Estudo de caso na Microsoft (ESEC/FSE 2023) de 23 bots de desenvolvimento que atuam nos pull requests do ambiente de CI/CD (análise estática, segurança, configuração, revisores, testes flaky): entrevistas com 13 autores de bots, 2 da plataforma e 9 usuários, survey com 54 usuários e logs de ~1,5 ano de ações dos bots. Os usuários relatam que os bots evitam PRs de correção, tiram do revisor o trabalho de apontar miudezas, orientam novatos sem ocupar um sênior e respondem dúvidas de privacidade que antes esperavam um dia; em contrapartida, muitos bots simultâneos, recomendações longas, sem próximo passo claro ou irrelevantes poluem a revisão e são ignoradas. A prática estudada (bots acoplados ao pipeline de CI/CD, em nome do shift-left) não cabe bem na tabela e vai como prática nova; os bots de segurança poderiam ser lidos como DevSecOps, mas os efeitos relatados não são específicos deles. Os dados de engajamento (RQ3) são só técnicos e não geram efeito.

Bots de desenvolvimento no pipeline de CI/CD ↑ **aumenta** Integração de novatos (fator +); Bots de desenvolvimento no pipeline de CI/CD ↕ **ambivalente** Carga de trabalho (fator −); Bots de desenvolvimento no pipeline de CI/CD ↑ **aumenta** Carga cognitiva (fator −); Bots de desenvolvimento no pipeline de CI/CD ↑ **aumenta** Documentação interna (fator +); Bots de desenvolvimento no pipeline de CI/CD ↑ **aumenta** Conflito de papéis (fator −)

Soltren e Mulligan (2023) — Enabling Pre-Merge CI on your TV

Proceedings - 2023 IEEE 16th International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2023

O PDF é só o resumo estendido (1 página) de uma palestra no ICSTW 2023: relato de experiência da Roku sobre migração de Perforce para git com submódulos e adoção de CI pré-merge obrigatório no firmware RokuOS, sem seção de método e sem nenhum dado sobre pessoas. O único dado é técnico: a taxa de builds quebrados caiu de 8,70% para 4,85% em três meses. O efeito sobre quem trabalha (desenvolvimento parado para desfazer mudanças antes do CI pré-merge; processo adicional para quem faz commit quando o merge falha) é narrado pelos próprios autores, por isso os elos são fracos (indireta e inferida).

Integração contínua ↓ **reduz** Manutenção do ambiente de desenvolvimento (fator −); Integração contínua ↑ **aumenta** Carga de trabalho (fator −)

Parry et al. (2022) — Surveying the Developer Experience of Flaky Tests

Proceedings - International Conference on Software Engineering

Survey online com 170 desenvolvedores (Likert de impactos, causas e ações, mais análise temática das respostas abertas) e análise temática de 38 threads do StackOverflow sobre testes instáveis (flaky). Os participantes concordam que testes instáveis atrapalham a CI e causam perda de produtividade; a ação mais comum é reexecutar o build, e quem convive com eles ao menos mensalmente tende mais a não agir e a ignorar falhas possivelmente genuínas. As respostas abertas trazem raiva e frustração (SQ11t2, SQ8t1), que os autores leem como custo psicológico, e apontam diferenças entre o ambiente local e o de CI como causa de instabilidade. O estudo não mede bem-estar; o elo é com o atrito de ferramenta e infraestrutura.

Automação de testes ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −); Pipeline de CI/CD ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Shetty et al. (2022) — AutoTSG: learning and synthesis for incident troubleshooting

ESEC/FSE 2022 - Proceedings of the 30th ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering

Estudo industrial na Microsoft (ESEC/FSE 2022; o PDF lido é a versão arXiv) sobre Troubleshooting Guides (TSGs, runbooks de incidente) usados por engenheiros de plantão: mineração de cliques ligando ~4800 TSGs a incidentes em 4 meses, codificação aberta de 428 feedbacks sobre TSGs, avaliação técnica da ferramenta AutoTSG e entrevistas curtas com 10 engenheiros de on-call. Incidentes sev 1–2 com TSG têm TTM médio menor (≈13 h contra ≈19 h), que os autores tomam como proxy do esforço de plantão; ao mesmo tempo, TSGs incompletos, desatualizados e confusos fazem o plantonista recorrer a colegas e verificar passos à mão. A parte sobre automação de TSGs (AutoTSG) é só percepção prospectiva dos entrevistados ("automating TSGs would help"), sem uso real, e não gera efeito.

Gestão de incidentes ↓ **reduz** Carga de trabalho (fator −); Gestão de incidentes ↑ **aumenta** Interrupções no trabalho (fator −); Gestão de incidentes ↑ **aumenta** Carga cognitiva (fator −)

Hemon et al. (2018) — Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams

International Conference on Information Systems 2018, ICIS 2018

Estudo de caso qualitativo embutido (ICIS 2018, literatura branca; versão depositada no HAL) numa grande empresa europeia de serviços: 54 entrevistas em profundidade com cinco papéis (DM, PO, AR, PE, RM) em 11 projetos, uns em modo ágil e outros em modo DevOps, mais a fase exploratória, com análise temática. Conclui que DevOps traz mais fontes de satisfação que o ágil (dinâmica de grupo, processo colaborativo, papéis de Ops reconhecidos), mas também insatisfações e riscos específicos: papéis PO/RM expostos a crítica, falta de feedback da hierarquia, sobrecarga dos operadores entre projeto e incidente, riscos escondidos pela automação. A comparação ágil × DevOps é por presença de temas, sem teste estatístico, e 8 dos 10 fatores de satisfação são comuns aos dois modos. Sobre a versão estendida (Hemon-Hildgen et al. 2020, EJIS, sem PDF): este texto basta para os 7 elos registrados, todos com fala de participante ou tabela; mas a parte de automação é curta (um risco e uma fala) e a extensão deve aprofundar automação, compartilhamento e condições de trabalho, então vale obtê-la para refinar mecanismos e condições, sem ser indispensável para os elos.

Times multifuncionais Dev–Ops ↑ **aumenta** Relações no trabalho (fator +); Times multifuncionais Dev–Ops ↕ **ambivalente** Reconhecimento no trabalho (fator +); Times multifuncionais Dev–Ops ↓ **reduz** Apoio do gestor (fator +); Times multifuncionais Dev–Ops ↑ **aumenta** Conflito de papéis (fator −); Implantação contínua ↑ **aumenta** Carga cognitiva (fator −); Releases frequentes ↑ **aumenta** Pressão de prazos (fator −); ESC/DevOps em geral ↑ **aumenta** Desenho do trabalho (fator −, nome neutro*)

Limoncelli (2018) — SQL is no excuse to avoid DevOps

Queue

AVISO: literatura cinza de 1º nível (coluna de opinião no ACM Queue, sem revisão por pares nem método). Thomas Limoncelli, gerente de SRE, defende que banco SQL não impede DevOps e descreve duas técnicas: atualização automática de esquema gerida pela aplicação e código compatível com vários esquemas (expand/contract), que permitem implantar sem janela de manutenção. O único dado é o relato de um amigo sobre o processo manual (implantação de horas no fim de semana, principal fonte de estresse, medo de perder o emprego); o efeito da automação sobre as pessoas é afirmação do autor a partir da experiência profissional, sem caso observado depois da adoção. Por isso os 2 elos são “inferida”, não “empírica indireta”: não há dado em que prática e fator apareçam juntos, só o estado anterior à prática e a previsão do autor.

Entrega contínua ↓ **reduz** Conflito trabalho-família (fator −); Entrega contínua ↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas)

Zhang et al. (2018) — One size does not fit all: An empirical study of containerized continuous deployment workflows

ESEC/FSE 2018 - Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering

Estudo de métodos mistos (ESEC/FSE 2018): survey com 168 donos de projetos OSS que publicam imagens Docker (codificação aberta de motivações, necessidades e barreiras) seguido de regressões de efeitos mistos sobre 855 projetos comparando o workflow de auto-builds do Docker Hub (DHW) com o workflow via serviço de CI (CIW). As regressões medem só desfechos técnicos (frequência de release, builds com erro, estabilidade do Dockerfile, latência); o elo com pessoas vem das falas do survey: a implantação contínua tira do desenvolvedor o trabalho manual de construir e publicar, e o pipeline ao mesmo tempo poupa e cria trabalho de configuração, manutenção e espera de build. Os autores tratam o modo de adoção como trade-off: CIW é mais configurável e confiável, mas tem mais builds com erro e exige mais experiência; DHW é mais simples e de manutenção mais baixa.

Implantação contínua ↓ **reduz** Carga de trabalho (fator −); Pipeline de CI/CD ↕ **ambivalente** Manutenção do ambiente de desenvolvimento (fator −)

Claes et al. (2017) — Abnormal Working Hours: Effect of Rapid Releases and Implications to Work Content

IEEE International Working Conference on Mining Software Repositories

Mineração de repositórios (MSR 2017): timestamps de 5,86 milhões de comentários de patch e revisão no Bugzilla do Mozilla Firefox, comparando a proporção de atividade em fins de semana e à noite antes e depois da passagem, em 2011, de releases anuais para releases a cada 6 semanas (e o Firefox com o Firefox OS). Depois das releases rápidas a proporção de trabalho fora do horário comercial caiu, com efeito grande nos fins de semana e menor à noite; restringindo aos contratados da Mozilla, a queda nos fins de semana se mantém e a da noite fica desprezível. Os autores concluem que a suposição de que mais prazos geram mais pressão e mais trabalho fora de hora não se confirmou. O trabalho fora de hora é medido por proxy (horário dos comentários), numa só organização, e o estudo não mede pressão de prazos nem bem-estar diretamente.

Releases frequentes ↓ **reduz** Conflito trabalho-família (fator −)

Pinto et al. (2017) — Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users

Proceedings - 2017 IEEE/ACM 10th International Workshop on Cooperative and Human Aspects of Software Engineering, CHASE 2017

Survey online (CHASE 2017) com 158 desenvolvedores que já quebraram ao menos um build em projetos OSS populares que usam Travis CI, com análise qualitativa das respostas abertas sobre causas de quebra de build e benefícios e problemas da integração contínua. Do lado das pessoas, os respondentes relatam que a CI dá confiança para mudar o código, mas cobra configuração e manutenção do ambiente de build, disciplina sob a norma de que o master tem de compilar, esforço extra, deslocamento do trabalho para a infraestrutura de build e dificuldade para novatos. A pressão de tempo aparece como causa de quebra de build, não como efeito da CI, e por isso não foi registrada.

Pipeline de CI/CD ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −); Integração contínua ↑ **aumenta** Carga de trabalho (fator −); Integração contínua ↑ **aumenta** Pressão por manter o build verde (fator emergente, Tarefa e carga de trabalho); Integração contínua ↓ **reduz** Alinhamento do uso do tempo (fator +); Integração contínua ↓ **reduz** Integração de novatos (fator +); Integração contínua ↑ **aumenta** Confiança para alterar o código (fator emergente, Técnico e ferramentas)

Souza e Silva (2017) — Sentiment Analysis of Travis CI Builds

IEEE International Working Conference on Mining Software Repositories

Estudo de mineração de repositórios (MSR Challenge) com 1.262 projetos Java e Ruby do TravisTorrent (609 mil builds, 1 milhão de commits), com sentimento das mensagens de commit medido pelo SentiStrength ajustado e testes de qui-quadrado e Mann-Whitney. Commits seguintes a um build quebrado no servidor de CI são um pouco mais negativos e menos positivos, mas com efeito muito pequeno (Cliff's delta −0,014), e os autores avisam que o efeito real pode ser ainda menor por imprecisão da ferramenta. O sentido inverso (sentimento negativo → build quebrado) é pessoa → pipeline e não gerou registro; commits que mencionam o Travis CI são levemente mais positivos, possivelmente por sarcasmo, sem elo de prática. Nenhum fator intermediário é medido: o registro é direto ao desfecho.

Pipeline de CI/CD ↑ **aumenta** *desfecho:* sentimento negativo expresso nas mensagens de commit

---

Itkonen et al. (2016) — Perceived Benefits of Adopting Continuous Delivery Practices

International Symposium on Empirical Software Engineering and Measurement

Estudo de caso exploratório (entrevistas semiestruturadas com três desenvolvedores do fornecedor e duas pessoas do cliente, análise temática) sobre cinco anos de transição para entrega contínua numa organização fornecedor–cliente que atende um cliente do setor público na Finlândia. Os desenvolvedores relatam menos tarefa manual repetitiva, feedback mais rápido, menor risco de falha na implantação (a que atribuem boa parte da queda do estresse) e menos retrabalho por especificação mal entendida; a adoção, por outro lado, custou esforço de aprendizado acima do previsto. Sobre o cliente: a colaboração não foi registrada como 'aumenta Interação com clientes', porque no catálogo esse fator é o desgaste (requisito que muda, cliente indisponível para tirar dúvida, trabalho feito virando trabalho perdido) e o artigo mostra o contrário: o contato cresceu e esse desgaste caiu. Por isso o elo é 'reduz Interação com clientes', com evidência indireta porque as práticas de colaboração vieram junto com o pipeline e o artigo não as separa; não criei fator emergente de colaboração porque o ganho do lado do desenvolvedor já cabe nesse registro e no de Desenho do trabalho, e duplicar inflaria a matriz. O cansaço de estar disponível todo dia foi relatado só pelo usuário-chave do cliente, que não produz software, e ficou de fora.

Entrega contínua ↓ **reduz** Interação com clientes (fator −, nome neutro*); Entrega contínua ↑ **aumenta** Desenho do trabalho (fator −, nome neutro*); Entrega contínua ↓ **reduz** Risco de falha na implantação (fator emergente, Técnico e ferramentas); Entrega contínua ↕ **ambivalente** Carga de trabalho (fator −)

Kärpänoja et al. (2016) — Exploring peopleware in continuous delivery

ACM International Conference Proceeding Series

Artigo de workshop (XP '16 Workshops, ACM), derivado de uma dissertação de mestrado: três entrevistas semiestruturadas de cerca de duas horas com projetistas líderes de três projetos web pequenos (4 a 7 pessoas) da Solita, todos com pipeline de entrega contínua automatizado e implantação em produção possível todo dia. Os autores relatam que a prontidão para implantar e corrigir rápido tira o estresse com bugs e implantações, que implantações automatizadas acontecem no horário normal sem hora extra, que a responsabilidade do time por testes, qualidade e infraestrutura dá mais controle e forma um time unido (para um entrevistado, também mais estresse pela exigência de competência mais ampla), que implantar pouco (projetos B e C) aumenta o receio da próxima implantação e a confusão entre versões, e que testes end-to-end em navegador falham sem motivo e consomem tempo. Limitações declaradas: amostra de três pessoas, entrevistas feitas por um único pesquisador que participou de um dos projetos, sem transcrição completa, e desenvolvedores descritos como abertos a mudança.

Entrega contínua ↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas); Releases frequentes ↓ **reduz** Ansiedade de implantação (fator emergente, Técnico e ferramentas); Releases frequentes ↓ **reduz** Carga cognitiva (fator −); Entrega contínua ↓ **reduz** Conflito trabalho-família (fator −); Entrega contínua ↑ **aumenta** Relações no trabalho (fator +); Responsabilidade de operação pelo time ↑ **aumenta** Desenho do trabalho (fator −, nome neutro*); Automação de testes ↑ **aumenta** Manutenção do ambiente de desenvolvimento (fator −)

Rossi et al. (2016) — Continuous deployment of mobile software at facebook (showcase)

Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering

Relato industrial (FSE Showcase) com análise quantitativa de 7 anos de dados do Facebook (commits, crashes, cherry-picks, issues de produção) enquanto o ciclo de release do Android caiu de 8 para 1 semana. Conclui que a frequência de deploy não altera produtividade (LoC e pushes por desenvolvedor) nem qualidade — testou e não achou efeito. O único elo com pessoas no corpo do texto é o da corrida para o dia do corte do release branch, que diminui com ciclos mais curtos; a afirmação de que a automação 'reduz a carga do desenvolvedor' aparece só no abstract, e o corpo diz apenas que melhorar ferramentas e automação foi a maior parte do trabalho. Ao mover o corte para domingo, os pushes no fim de semana subiram (iOS 0,3→0,5 e Android 0,175→0,5 por desenvolvedor/dia), mas os autores não discutem isso como efeito sobre as pessoas.

Releases frequentes ↓ **reduz** Pressão de prazos (fator −)

Aprovados sem elo no texto (1)

Fluri et al. (2023) — Measuring the Benefits of CI/CD Practices for Database Application Development
Proceedings - 2023 IEEE/ACM International Conference on Software and System Processes, ICSSP 2023
Artigo do ICSSP 2023 com dois estudos de caso industriais (Oracle; UC1-DWH com 20 desenvolvedores e UC2-BE com 7) em que os autores implantaram um pipeline de CI/CD, com métricas antes e depois e entrevistas de 90 min com dois desenvolvedores por caso, antes e depois; as entrevistas só são resumidas, sem falas literais. É o MESMO estudo e o MESMO dado de Fluri2024 (JSEP), que é a versão estendida: os dois casos são dois dos três de Fluri2024, os números da Tabela IV coincidem com as colunas correspondentes da Tabela 4 de lá e a análise qualitativa é praticamente o mesmo texto ("two use cases" em vez de "three"). Todos os elos que o texto sustenta (pipeline reduz carga cognitiva, reduz interrupções e troca de contexto, aumenta autonomia ao permitir que todos implantem, aumenta a confiança ao implantar) já estão registrados em Fluri2024; o que sobra aqui — hora extra por implantação fora do expediente antes do pipeline, sem medida depois, e janelas fixas de release obrigando a acompanhar a promoção das mudanças — também aparece lá e não sustenta elo novo. Por isso nenhum efeito é registrado aqui, para não contar a mesma evidência duas vezes na matriz.

Reprovados na leitura do texto completo (3)

Gummadi et al. (2026) — Service Level Objective (SLO) Observability with Splunk and Dynatrace in Microservices
2026 International Conference on Artificial Intelligence, Systems, and Emerging Technologies, ICAISET 2026
Motivo: pergunta 4 / sem dados próprios: texto conceitual e de recomendações; os efeitos sobre pessoas (fadiga de alertas, menos apagar incêndio) são afirmados com base em documentação de fornecedores e blogs citados.
Artigo conceitual de praticantes sobre observabilidade centrada em SLO com plataformas de mercado em microsserviços: conceitos de SLI/SLO e error budget, configuração das ferramentas, correlação de logs, métricas e traces, alertas guiados por SLO e boas práticas. Não descreve método, amostra nem coleta; os 'case studies' são relatos resumidos de blogs e fornecedores citados. As afirmações de que SLOs reduzem a fadiga de alertas e o trabalho repetitivo de apagar incêndio não vêm de dados do próprio estudo, então não há elo prática → fator a extrair.

Lertpongrujikorn et al. (2026) — Empirical Analysis of Cloud-Edge Infrastructure Complexity: Practitioner Pain Points and Architectural Directions
Proceedings - International Conference on Computer Communications and Networks, ICCCN
Motivo: veículo: preprint arXiv sem revisão (arXiv:2608.08400v1); além disso, pergunta 4 fraca: os dados não ligam uma prática de ESC a um fator.
Estudo de customer discovery (NSF I-Corps) com 101 entrevistas semiestruturadas em 86 organizações, codificadas em 11 categorias de dores e 13 de expectativas sobre infraestrutura cloud-edge. Os achados são frequências: complexidade de implantação (38,6%), dificuldade de onboarding (35,6%) e sobrecarga cognitiva pela co-ocorrência entre complexidade e onboarding; os participantes querem mais automação em pipelines de CI/CD e rollback, mas isso é expectativa, não efeito observado. A ligação engenharia de plataforma → redução de carga cognitiva vem de relatórios citados (Puppet 2023, Greiler et al. 2023), e a seção IV propõe direções arquiteturais sem dado próprio. Mesmo que o veículo fosse aceito, não haveria elo prática → fator evidenciado pelos dados.

Tennakoon e Jayatissa (2025) — Achieving Sustainability in Sri Lankan SME IT Firms through DevOps
Proceedings - International Research Conference on Smart Computing and Systems Engineering, SCSE 2025
Motivo: pergunta 4: a variável dependente é um índice agregado de sustentabilidade; efeito sobre pessoas só no abstract e nas hipóteses.
Survey com 136 profissionais de 42 PMEs de TI do Sri Lanka, analisado por PLS-SEM, com CI/CD, IaC, monitoramento e feedback e colaboração e cultura como preditores de um único construto de sustentabilidade ('Sus'), que mistura as dimensões ambiental, econômica e social. Todos os caminhos dão positivos (p entre 0,004 e 0,049), mas nenhum isola a dimensão social nem mede fator ou desfecho das pessoas. A satisfação, a retenção e a cultura de aprendizado contínuo citadas no abstract não aparecem nos resultados. O texto ainda se contradiz sobre H4 (aceita na Tabela V, 'not validated' na discussão).

Sem PDF (6)

Ferazza e Mersinas (2026) — Socio-technical Friction: An Emergent Grounded Theory of DevSecOps Challenges
Springer (LNCS 2026): sem acesso pela CAPES, só a prévia. No abstract: Teoria fundamentada com 18 entrevistas (9 gestores de cibersegurança e 9 engenheiros de software) sobre os obstáculos à adoção de DevSecOps. Conclui que os engenheiros vivem as iniciativas de segurança como disruptivas, ineficientes e prejudiciais à sua experiência de desenvolvedor, e que o desalinhamento sociotécnico (prioridades desalinhadas, comunicação quebrada) gera atrito.

Claassen e Boekhorst (2015) — Shorter feedback loops by means of continuous deployment
Springer (LNBIP 2015): sem acesso pela CAPES, só a prévia. No abstract: Artigo curto que propõe usar implantação contínua para fechar cedo o ciclo de feedback das funcionalidades. Afirma que a prática também diminui o esforço de implantação dos desenvolvedores, aumentando sua felicidade; o abstract não descreve método nem amostra.

Hemon-Hildgen et al. (2020) — Orchestrating automation and sharing in DevOps teams: a revelatory case of job satisfaction factors, risk and work conditions
Taylor & Francis (EJIS): sem acesso pela CAPES. No abstract: Estudo de caso com 59 funcionários de 12 times ágeis e DevOps de uma mesma empresa, à luz das teorias de características do trabalho e de Herzberg. Conclui que DevOps dá mais satisfação no trabalho que o ágil sozinho, mas amplifica riscos e exige orquestrar automação e compartilhamento, a depender das condições de trabalho.

Jothi et al. (2026) — Empowering Developer Productivity through Platform Engineering: A Transformative Approach to Scalable and Streamlined Software Development
Wiley (capítulo de livro 2026): sem acesso pela CAPES. No abstract: Capítulo de livro acadêmico, conceitual, que discute como a engenharia de plataforma (automação, fluxos padronizados, autoatendimento) melhora a produtividade e a experiência do desenvolvedor, citando redução da carga cognitiva. O abstract não traz dados nem resultados empíricos; a afirmação aparece como tema do capítulo.

Kirpitsas e Pachidis (2026) — KIBO, a new hybrid software development method with enhanced information systems auditing capability
Elsevier: acesso aberto, mas o PDF não baixou pelo navegador. No abstract: Propõe o KIBO, método híbrido que integra Scrum, DevOps e Disciplined Agile Delivery com auditoria contínua, avaliado por 90 especialistas e numa implantação de 7 semanas; o time relatou melhora de qualidade, confiabilidade do processo e satisfação líquida dos desenvolvedores. A satisfação é atribuída ao método como um todo, não a uma prática de ESC isolada.

Kulas et al. (2014) — Practical experience with test-driven development during commissioning of the multi-star AO system ARGOS
SPIE: acesso aberto, mas o PDF não baixou pelo navegador. No abstract: Relato de experiência do uso de TDD, programação em par e integração contínua por engenheiros de software durante o comissionamento do sistema ARGOS, fase de pressão de tempo e estresse. O abstract diz que a equipe conseguiu manter o TDD sob pressão, mas não traz resultados sobre o efeito da integração contínua sobre as pessoas.

6. Para o *snowballing*

Primários citados pelos artigos lidos que ligam ESC a pessoas, estudos secundários da busca 1, as sementes que nenhuma string alcança e os trabalhos citados pelo corpus da etapa 1.

Fonte	Indicado por / por quê
Ståhl e Bosch 2013	**Ahmad et al. (2023)**: Ståhl e Bosch 2013 — benefícios experimentados de CI em empresa, incluindo produtividade dos desenvolvedores
Hilton et al. 2016	**Ahmad et al. (2023)**: Hilton et al. 2016 — usos, custos e benefícios de CI em projetos open source, com survey de desenvolvedores
Leppänen et al. 2015	**Ahmad et al. (2023)**: Leppänen et al. 2015 — entrevistas sobre obstáculos e benefícios da implantação contínua (IEEE Software, cinza de 1º nível)
Kasi et al. 2008	**Anandayuvaraj et al. (2026)**: Kasi et al. 2008 — estudo Delphi ("post mortem paradox") sobre percepções de especialistas de TI: tempo, recursos e barreiras culturais no postmortem
Desouza, Dingsøyr e Awazu 2005	**Anandayuvaraj et al. (2026)**: Desouza, Dingsøyr e Awazu 2005 — experiências conduzindo postmortems de projeto (relatórios versus histórias)
Vieira et al. 2019	**Anandayuvaraj et al. (2026)**: Vieira et al. 2019 — dificuldades técnicas e gerenciais na análise postmortem em projetos de software
Sillito e Pope 2024	**Anandayuvaraj et al. (2026)**: Sillito e Pope 2024 (CHASE) — aprender com lições de incidentes; achados preliminares sobre aprendizado com falhas
Wessel et al. 2021	**Asthana et al. (2023)**: Wessel et al. 2021 — "Don't Disturb Me": ruído e desafios de interação com bots em pull requests de projetos open source
Brown e Parnin 2020	**Asthana et al. (2023)**: Brown e Parnin 2020 — arquitetura de escolha das recomendações de bots e incômodo para o desenvolvedor
Erlenhov et al. 2020	**Asthana et al. (2023)**: Erlenhov et al. 2020 — características e desafios de bots de desenvolvimento na perspectiva de praticantes
Hilton et al. 2017	**Asthana et al. (2023)**: Hilton et al. 2017 — trade-offs da integração contínua (garantia, segurança, flexibilidade) relatados por desenvolvedores
Fluri et al. 2023	**Fluri et al. (2024)**: Fluri et al. 2023 (ICSSP) — versão anterior do mesmo estudo, com dois dos casos; conferir sobreposição antes de incluir
Claps et al. 2015	**Fluri et al. (2024)**: Claps et al. 2015 — desafios técnicos e sociais na adoção de implantação contínua
Shahin et al. 2017	**Fluri et al. (2024)**: Shahin et al. 2017 (ESEM) — investigação empírica dos desafios de implantação contínua, inclusive de equipe e coordenação
Riungu-Kalliosaari et al. 2016	**Fluri et al. (2024)**: Riungu-Kalliosaari et al. 2016 — estudo de caso de benefícios e desafios da adoção de DevOps
Lwakatare et al. 2019	**Fluri et al. (2024)**: Lwakatare et al. 2019 — estudo de caso múltiplo de DevOps em cinco empresas
Shropshire e Sweeney 2017	**Hemon et al. (2018)**: Shropshire e Sweeney 2017 — DevOps e moral da força de trabalho; acharam satisfação em queda após a implantação de DevOps
Shropshire, Menard e Sweeney 2017	**Hemon et al. (2018)**: Shropshire, Menard e Sweeney 2017 — personalidade e atitudes diante de DevOps; aspectos negativos relatados: trabalho mais duro, mais responsabilidade, autoridade difusa, métricas complexas, medo de demissão
Nybom, Smeds e Porres 2016	**Hemon et al. (2018)**: Nybom, Smeds e Porres 2016 — impacto de misturar responsabilidades de Dev e Ops: colaboração, mas também nova fonte de atrito
Wiedemann 2017	**Hemon et al. (2018)**: Wiedemann 2017 — nova forma de colaboração em times de TI na adoção de DevOps (capital social)
Leppänen et al. 2015	**Itkonen et al. (2016)**: Leppänen et al. 2015 — The Highways and Country Roads to Continuous Deployment (IEEE Software); estudo com empresas sobre adoção de implantação contínua, citado por identificar a colaboração estreita entre desenvolvimento, operação e teste

Fonte	Indicado por / por quê
Neely e Stolt 2013	**Itkonen et al. (2016)**: Neely e Stolt 2013 — Continuous Delivery? Easy! Just Change Everything (Agile Conference); relato de experiência de adoção de CD, citado pelo mesmo motivo (cinza de 1º nível, entra com aviso se entrar)
Wiedemann et al. 2019	**Jeyam et al. (2026)**: Wiedemann et al. 2019 — citado como fonte de que plantão e disponibilidade constante geram tecno-invasão e de que a evolução de ferramentas gera tecno-incerteza em times DevOps (ICIS)
Sen 2021	**Jeyam et al. (2026)**: Sen 2021 — citado como fonte de que o ritmo dos pipelines de CI/CD e a automação levam a tecno-sobrecarga quando a carga cresce sem apoio (capítulo de livro)
Rau 2016	**Kafoe et al. (2026)**: Rau 2016 (capítulo 'Eliminating Toil' em Beyer et al., Site Reliability Engineering, O'Reilly) — afirma que reduzir toil aumenta satisfação e que acúmulo de toil causa baixa moral e estagnação de carreira; livro, cinza de 1º nível
Lwakatare et al. 2019	**Kafoe et al. (2026)**: Lwakatare et al. 2019 — DevOps in Practice: estudo de caso múltiplo em cinco empresas; curva de aprendizado íngreme de habilidades operacionais em DevOps
Puppet Labs 2015	**Kafoe et al. (2026)**: Puppet Labs 2015 — State of DevOps Report; relatório de survey ligando DevOps a produtividade e qualidade, cinza
Ziftci e Cavalcanti 2020	**Kassab et al. (2026)**: Ziftci e Cavalcanti 2020 — citado para testes flaky que minam a confiança na automação e geram reexecuções e frustração (provavelmente estudo técnico de causa raiz no Google; conferir se há dado sobre pessoas)
Neely e Stolt 2013	**Kärpänoja et al. (2016)**: Neely e Stolt 2013 (AGILE) — estudo de caso da Rally Software sobre a adoção de entrega contínua pelos engenheiros, com resistência à mudança e o estresse dos "scary Saturdays"
Chen 2015	**Kärpänoja et al. (2016)**: Chen 2015 (IEEE Software) — benefícios e desafios da entrega contínua relatados na prática (cinza de 1º nível)
Puppet 2023	**Lertpongrujikorn et al. (2026)**: Puppet 2023 (State of Platform Engineering Report) — relatório de indústria (cinza) usado pelo artigo para ligar engenharia de plataforma à redução da carga cognitiva dos times
Greiler, Storey e Noda 2023	**Lertpongrujikorn et al. (2026)**: Greiler, Storey e Noda 2023 (DevEx, ACM Queue) — fluxo, ciclos de feedback e carga cognitiva como dimensões da experiência do desenvolvedor; candidato a conferir se liga práticas de entrega a esses fatores
Gruber e Fraser 2022	**Parry et al. (2022)**: Gruber e Fraser 2022 — survey sobre como a instabilidade de testes afeta os desenvolvedores e que apoio eles precisam
Eck et al. 2019	**Parry et al. (2022)**: Eck et al. 2019 — perspectiva de desenvolvedores da Mozilla sobre testes instáveis (classificação e survey com 121 respostas)
Hilton et al. 2017	**Parry et al. (2022)**: Hilton et al. 2017 — survey na Pivotal sobre trade-offs da CI, inclui builds falhando por testes instáveis
Hilton et al. 2016	**Pinto et al. (2017)**: Hilton et al. 2016 — uso, custos e benefícios da integração contínua em projetos open source, com survey de desenvolvedores
Claes et al. 2017	**Riedl et al. (2025)**: Claes et al. 2017 — rapid releases e horário anormal de trabalho (Releases frequentes → jornada)
Müller e Benlian 2022	**Riedl et al. (2025)**: Müller e Benlian 2022 — uso de práticas ágeis de desenvolvimento (teste unitário, refatoração, padrões de código) → fadiga, carga percebida e intenção de sair; verificar se separa CI
Benlian 2022	**Riedl et al. (2025)**: Benlian 2022 — uso de práticas ágeis de ISD → bem-estar diário (avaliação como desafio ou ameaça); verificar se inclui CI
Venkatesh et al. 2020	**Riedl et al. (2025)**: Venkatesh et al. 2020 — uso de métodos ágeis (instrumento de Maruping, com CI) → exaustão via ambiguidade e conflito de papel; verificar resultado por prática
Meier et al. 2018	**Riedl et al. (2025)**: Meier et al. 2018 — estresse no desenvolvimento ágil por prática; verificar se analisa CI

Fonte	Indicado por / por quê
Savor et al. 2016	**Rossi et al. (2016)**: Savor et al. 2016 — implantação contínua no Facebook e na OANDA, com desenvolvedores responsáveis por testar e implantar o próprio código; primário dos mesmos autores, pode ter relato sobre as pessoas
Jiang et al. 2020	**Shetty et al. (2022)**: Jiang et al. 2020 — recomendação de TSGs e efeito na redução do esforço de mitigação de incidentes por engenheiros de plantão
Li et al. 2021	**Shetty et al. (2022)**: Li et al. 2021 — dificuldade dos engenheiros de plantão para mitigar incidentes em serviços com dependências complexas
Díaz et al. 2018	**Tennakoon e Jayatissa (2025)**: Díaz et al. 2018 — estudo de caso exploratório de DevOps na prática; pode trazer efeitos de cultura e colaboração sobre os times
Perera et al. 2016	**Tennakoon e Jayatissa (2025)**: Perera et al. 2016 — impacto da prática de DevOps em organizações de software do Sri Lanka; primário, a conferir se trata de pessoas
Rombaut et al. 2023	**Valenzuela-Toledo et al. (2024)**: Rombaut et al. 2023 — overhead de manutenção que o bot de atualização de dependências (Greenkeeper) gera para os desenvolvedores
McIntosh et al. 2011	**Valenzuela-Toledo et al. (2024)**: McIntosh et al. 2011 — esforço de manutenção de arquivos de build, artefato análogo ao pipeline
Saroar e Nayebi 2023	**Valenzuela-Toledo et al. (2024)**: Saroar e Nayebi 2023 — survey com desenvolvedores sobre percepção, motivações e desafios do GitHub Actions
Zhao et al. 2017	**Valenzuela-Toledo et al. (2024)**: Zhao et al. 2017 — impacto da adoção de CI sobre as demais práticas de desenvolvimento
Neely e Stolt 2013	**Zhang et al. (2018)**: Neely e Stolt 2013 — relato de adoção de entrega contínua citado por menos trabalho fora de hora e mais confiança
Claps, Berntsson Svensson e Aurum 2015	**Zhang et al. (2018)**: Claps, Berntsson Svensson e Aurum 2015 — desafios técnicos e sociais na transição para implantação contínua
Chen 2015	**Zhang et al. (2018)**: Chen 2015 — benefícios e desafios da entrega contínua numa organização
Hilton et al. 2017	**Zhang et al. (2018)**: Hilton et al. 2017 — trade-offs da integração contínua (garantia, segurança, flexibilidade) com entrevistas a desenvolvedores
Widder et al. 2018	**Zhang et al. (2018)**: Widder et al. 2018 — por que projetos abandonam o Travis CI
Pérez-Sánchez et al. (2024) A Taxonomy on Human Factors that Affect DevOps Adoption	Mapeamento sistemático (21 estudos) que cria uma taxonomia de 59 fatores humanos que afetam a adoção de DevOps, com foco em cultura. A direção é pessoas → adoção, não prática → pessoas; candidato a snowballing pelos primários que tratam de cultura e fatores humanos em DevOps.
Lunde e Colomo-Palacios (2020) Continuous practices and technical debt: A systematic literature review	Revisão sistemática sobre como práticas contínuas e DevOps afetam a dívida técnica (fator do catálogo), com impactos positivos e negativos; conclui que podem reduzi-la se bem aplicadas. Não trata de pessoas diretamente, mas os primários sobre práticas contínuas e dívida técnica podem servir ao snowballing.
Zimmermann (2017) Microservices tenets: Agile approach to service development and deployment	Artigo de visão geral que revisa introduções populares a microsserviços e os compara com SOA; afirma que microsserviços e os paradigmas associados (inclusive a abordagem DevOps contínua) aumentam a carga cognitiva dos arquitetos. Não é estudo primário e o efeito é atribuído a microsserviços, não a uma prática de ESC.
Jha et al. (2023) From theory to practice: Understanding DevOps culture and mindset	Revisão sistemática sobre cultura e mentalidade DevOps (princípios, práticas, ferramentas, desafios), que diz ajudar a entender o fenômeno pela perspectiva de fatores humanos e identifica três temas culturais: colaboração, melhoria contínua e automação. Candidato a snowballing pelos primários sobre cultura DevOps.
Pérez-Sánchez et al. (2025) A theory on human factors in DevOps adoption	Mapeamento sistemático seguido de clusterização e avaliação por questionário (n=15) que identifica 59 fatores humanos relacionados ao DevOps e propõe uma teoria sobre fatores humanos na adoção de DevOps; a direção estudada é fatores humanos → adoção, não DevOps → pessoas. Candidato a snowballing pelos primários resenhados.
Laukkanen e Mantyla (2015) Build Waiting Time in Continuous Integration-An Initial Interdisciplinary Literature Review	Revisão interdisciplinar da literatura sobre tempo de espera do build em integração contínua; classifica os efeitos da espera em específicos de CI, cognitivos e emocionais e sugere reduzir a percepção da espera com feedback e outras atividades durante a integração. Candidato a snowballing pelos primários sobre efeitos cognitivos e emocionais da espera do build.

Fonte	Indicado por / por quê
Westrum (2004), *A typology of organisational cultures*	Base da ligação práticas → cultura generativa citada por Trinkenreich et al. (2023)
Relatórios DORA / *Accelerate*	Dor do *deploy*, retrabalho e *burnout* associados às capacidades de entrega contínua
Hilton et al. (2017); Fagerholm et al. (2017); Greiler et al. (2022)	Sementes sem prática de ESC no resumo; nenhuma string as recupera

7. Referências (candidatos da busca 1)

Ahmad, Azeem; Neto, Francisco Gomes De Oliveira; Enoiu, Eduard Paul; Sandahl, Kristian; Leifler, Ola (2023). An Industrial Study on the Challenges and Effects of Diversity-Based Testing in Continuous Integration. *IEEE International Conference on Software Quality, Reliability and Security, QRS*. DOI: 10.1109/QRS60937.2023.00041

Anandayuvaraj, Dharun; Singla, Tanmay; Hammadeh, Zain Alabedin Haj; Lund, Andreas; Holloway, Alexandra; Davis, James C. (2026). Learning from Software Failures: A Case Study at a National Space Research Center. *Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering, ICSE 2026*. DOI: 10.1145/3744916.3773149

Andreyev, Oleg (2025). The Art of Managing Flaky Tests at Scale. *2025 IEEE International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2025*. DOI: 10.1109/ICSTW64639.2025.10962486

Asthana, Sumit; Sajnani, Hitesh; Voyloshnikova, Elena; Acharya, Birendra; Herzig, Kim (2023). A Case Study of Developer Bots: Motivations, Perceptions, and Challenges. *ESEC/FSE 2023 - Proceedings of the 31st ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. DOI: 10.1145/3611643.3616248

Claassen, Arjan; Boekhorst, Laurens (2015). Shorter feedback loops by means of continuous deployment. *Lecture Notes in Business Information Processing*. DOI: 10.1007/978-3-319-18612-2_24

Claes, Maelick; Mantyla, Mika; Kuutila, Miikka; Adams, Bram (2017). Abnormal Working Hours: Effect of Rapid Releases and Implications to Work Content. *IEEE International Working Conference on Mining Software Repositories*. DOI: 10.1109/MSR.2017.3

Ferazza, Francesco; Mersinas, Konstantinos (2026). Socio-technical Friction: An Emergent Grounded Theory of DevSecOps Challenges. *Lecture Notes in Computer Science*. DOI: 10.1007/978-3-032-20018-1_22

Fluri, Jasmin; Fornari, Fabrizio; Pustulka, Ela (2023). Measuring the Benefits of CI/CD Practices for Database Application Development. *Proceedings - 2023 IEEE/ACM International Conference on Software and System Processes, ICSSP 2023*. DOI: 10.1109/ICSSP59042.2023.00015

Fluri, Jasmin; Fornari, Fabrizio; Pustulka, Ela (2024). On the importance of CI/CD practices for database applications. *Journal of Software: Evolution and Process*. DOI: 10.1002/smr.2720

Gummadi, Venkata Pavan Kumar; Chilamkurthi, Lakshmi Sujatha; Kavuri, Srikanth (2026). Service Level Objective (SLO) Observability with Splunk and Dynatrace in Microservices. *2026 International Conference on Artificial Intelligence, Systems, and Emerging Technologies, ICAISET 2026*. DOI: 10.1109/ICAISET66439.2026.11541542

Hemon, Aymeric; Monnier-Senicourt, Laetitia; Rowe, Frantz (2018). Job satisfaction factors and risks perception: An embedded case study of DevOps and Agile Teams. *International Conference on Information Systems 2018, ICIS 2018*.

Hemon-Hildgen, Aymeric; Rowe, Frantz; Monnier-Senicourt, Laetitia (2020). Orchestrating automation and sharing in DevOps teams: a revelatory case of job satisfaction factors, risk and work conditions. *European Journal of Information Systems*. DOI: 10.1080/0960085X.2020.1782276

Itkonen, Juha; Udd, Raoul; Lassenius, Casper; Lehtonen, Timo (2016). Perceived Benefits of Adopting Continuous Delivery Practices. *International Symposium on Empirical Software Engineering and Measurement*. DOI: 10.1145/2961111.2962627

Jeyam, Dharneeka; Wiedemann, Anna; Schwabe, Gerhard; Güney, Kadircan (2026). "still in the Loop": Coping with Technostress in DevOps Teams and the Impact of GenAI. *Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP 2026*. DOI: 10.1145/3786583.3786899

Jothi, C. Saranya; Surya, E.; Rabiya, M. Syed; Lalitha, B.; Roselinkiruba, R. (2026). Empowering Developer Productivity through Platform Engineering: A Transformative Approach to Scalable and Streamlined Software Development. *Platform Engineering: Concepts, Challenges, and Applications*. DOI: 10.1002/9781394395910.ch1

Kafoe, Tom; Ochoa, Lina; Siravuru, Sharath; Serebrenik, Alexander (2026). "making Our Life Less Monotonous" or "just Tick Things Off": An Exploratory Multi-Method Study of Toil. *Proceedings - 2026 IEEE/ACM 48th International Conference on Software Engineering, ICSE 2026*. DOI: 10.1145/3744916.3764540

Kassab, Mohamad; Sohail, Izah; Fernandes, João Paulo (2026). The Emotional Cost of Technical Debt: Quantifying Developer Frustration Across Debt-Related Concern Themes. *FSE Companion 2026 - Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. DOI: 10.1145/3803437.3805542

Kirpitsas, Ioannis; Pachidis, Theodore (2026). KIBO, a new hybrid software development method with enhanced information systems auditing capability. *Information and Software Technology*. DOI: 10.1016/j.infsof.2025.107958

Kulas, M.; Borelli, Jose Luis; Gäsler, Wolfgang; Peter, Diethard; Rabien, Sebastian; Orban De Xivry, Gilles; Busoni, Lorenzo; Bonaglia, Marco; Mazzoni, Tommaso; Rahmer, Gustavo (2014). Practical experience with test-driven development during commissioning of the multi-star AO system ARGOS. *Proceedings of SPIE - The International Society for Optical Engineering*. DOI: 10.1117/12.2056218

Kärpänoja, Pauli; Virtanen, Antti; Lehtonen, Timo; Mikkonen, Tommi (2016). Exploring peopleware in continuous delivery. *ACM International Conference Proceeding Series*. DOI: 10.1145/2962695.2962708

Lertpongrujikorn, Pawissanutt; Nguyen, Hai Duc; Salehi, Mohsen Amini (2026). Empirical Analysis of Cloud-Edge Infrastructure Complexity: Practitioner Pain Points and Architectural Directions. *Proceedings - International Conference on Computer Communications and Networks, ICCCN*. DOI: 10.1109/ICCCN69946.2026.11662594

Limoncelli, Thomas A. (2018). SQL is no excuse to avoid DevOps. *Queue*. DOI: 10.1145/3291276.3300018

Parry, Owain; Kapfhammer, Gregory M.; Hilton, Michael; Mcminn, Phil (2022). Surveying the Developer Experience of Flaky Tests. *Proceedings - International Conference on Software Engineering*. DOI: 10.1109/ICSE-SEIP55303.2022.9793965

Pinto, Gustavo; Reboucas, Marcel; Castor, Fernando (2017). Inadequate testing, time pressure, and (Over) confidence: A tale of continuous integration users. *Proceedings - 2017 IEEE/ACM 10th International Workshop on Cooperative and Human Aspects of Software Engineering, CHASE 2017*. DOI: 10.1109/CHASE.2017.13

Riedl, René; Oettl, Christian; Stangl, Fabian J.; Hevner, Alan R. (2025). How an Agile Software Process Increases Developers' Job Satisfaction: A Stress Perspective Based on the Effort-Reward-Imbalance Model. *Business and Information Systems Engineering*. DOI: 10.1007/s12599-024-00919-x

Rossi, Chuck; Shibley, Elisa; Su, Shi; Beck, Kent; Savor, Tony; Stumm, Michael (2016). Continuous deployment of mobile software at facebook (showcase). *Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering*. DOI: 10.1145/2950290.2994157

Shetty, Manish; Bansal, Chetan; Upadhyayula, Sai Pramod; Radhakrishna, Arjun; Gupta, Anurag (2022). AutoTSG: learning and synthesis for incident troubleshooting. *ESEC/FSE 2022 - Proceedings of the 30th ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. DOI: 10.1145/3540250.3558958

Soltren, Jose; Mulligan, Kyle (2023). Enabling Pre-Merge CI on your TV. *Proceedings - 2023 IEEE 16th International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2023*. DOI: 10.1109/ICSTW58534.2023.00028

Souza, Rodrigo; Silva, Bruno (2017). Sentiment Analysis of Travis CI Builds. *IEEE International Working Conference on Mining Software Repositories*. DOI: 10.1109/MSR.2017.27

Tennakoon, Hiruni; Jayatissa, Yehemini (2025). Achieving Sustainability in Sri Lankan SME IT Firms through DevOps. *Proceedings - International Research Conference on Smart Computing and Systems Engineering, SCSE 2025*. DOI: 10.1109/SCSE65633.2025.11031052

Valenzuela-Toledo, Pablo; Bergel, Alexandre; Kehrer, Timo; Nierstrasz, Oscar (2024). The Hidden Costs of Automation: An Empirical Study on GitHub Actions Workflow Maintenance. *Proceedings - 2024 IEEE International Conference on Source Code Analysis and Manipulation, SCAM 2024*. DOI: 10.1109/SCAM63643.2024.00029

Zhang, Yang; Vasilescu, Bogdan; Wang, Huaimin; Filkov, Vladimir (2018). One size does not fit all: An empirical study of containerized continuous deployment workflows. *ESEC/FSE 2018 - Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. DOI: 10.1145/3236024.3236033